

Compiling Recurrent Agent Workflows into Guarded Programs

Artifact. Implementation, pinned data snapshots, raw results, generated figures and tables, and an adversarial self-review accompany this manuscript. **Evidence.** A ten-benchmark audit with two compiler substrates and six pilot-separated live-provider protocols on real public GitHub records.

Abstract

Tool-using agents often repeat mature, read-only evidence paths while paying for a model decision at every boundary. Replacing those decisions with deterministic programs can reduce cost, but recurrence alone is unsafe: arguments may lack observable provenance, tools may cross effect or permission boundaries, and a replay-correct prefix may still change the final answer.

We introduce *guarded agentic compaction* (GAC), a trace-to-program compiler that reconstructs typed value provenance, rejects unsafe effects, synthesizes from a closed 23-operator language, verifies empirical contracts, and applies exact finite-sample admission. Its normal output is refusal: compile only the supported prefix, otherwise retain the unchanged agent. Guarded composite synthesis can expose an admitted program before the first model request without discarding its internal checks.

On 132 discovery traces from real public GitHub issues, GAC rejects an ungroundable third call and emits a two-read prefix. On 30 held-out issues it preserves all exact task contracts while reducing provider requests by 50.0% and estimated cost by 32.0%; a hand-written macro also passes 30/30 and is cheaper. In a separate exploratory cohort, the pre-model composite matches a provider-visible macro on 12/12 exact contracts while halving requests; a fair pre-model manual baseline later ties it on six cases. On NESTFUL and API-Bank, every recurrent family is retired for insufficient support. A pinned audit of ten agent benchmarks finds only these two trace-complete compiler substrates.

The central result is an evidence boundary: traces establish recurrence, not admissibility. Agent workflow optimizers should be evaluated by what they safely refuse as well as by what they accelerate.

Keywords. LLM agents · tool use · workflow optimization · program synthesis · provenance · selective prediction · risk control · OpenAI Agents SDK

1 Introduction

The standard reasoning-acting loop delegates every tool boundary to a language model: model, tool, model, tool, and so on [48]. This flexibility is valuable while a workflow is novel. At scale, however, mature agents often revisit the same read-only evidence-gathering prefix with the same data dependencies. Repeating a model decision then consumes latency, tokens, and money without necessarily adding useful adaptation.

Deleting such decisions is not ordinary caching. A tool argument may depend on a prior observation, a repeated sequence may cross an approval or write, a nominal read

may have a hidden external effect, and an input that resembles the training trace may violate a permission or freshness constraint. A fast wrong workflow is worse than a slow agent. The research problem is therefore:

Given historical executions of a tool-using agent, identify a deterministic region that can replace model-mediated control flow, and dispatch it only where observable evidence supports that replacement.

This problem differs from current-query scheduling in LLMCompiler [24], prompt-program optimization in DSPy [23, 36], graph compression in AgentSlimming [9], model routing [33], and prompt compression [20]. It is closest to Agent Workflow Optimization (AWO), which converts frequent trace sequences into deterministic meta-tools [1], and to plan caching [50]. Our focus is the missing admission argument: typed value provenance, effect barriers, compatibility pins, post-execution verification, and finite-sample selective risk control.

We call the resulting approach *guarded agentic compaction* (GAC): a guarded *specialization* that compiles the routine part of a workflow — the recurrent read-only evidence prefix — while leaving every decision that the evidence does not determine to the model. Compaction names the mechanism, specialization names what it is allowed to do, and the boundary between them is the paper’s subject. The shipped implementation contains two passes over a common typed trace representation: guarded region compilation (GRC), studied here, and trace-guided workflow specialization (TGWS), which routes entry states to smaller prompt/tool configurations. We isolate GRC because it changes execution semantics more directly and is the contribution with new real-scenario evidence.

Our research questions are:

- RQ1.** Can typed trace provenance reconstruct the dependencies needed to synthesize recurrent tool programs?
- RQ2.** On an unseen real-record workload, how does a learned compiled prefix trade factual task quality against requests, tokens, latency, and cost relative to an unchanged agent and a hand-written composite tool, and can a guarded compiler close the interface gap exposed by that comparison when a fair pre-model manual program and a learned prompt optimizer receive the same held-out workload?
- RQ3.** Does exact selective admission reject recurrent families when calibration support is inadequate?
- RQ4.** Which failure modes remain after structural compilation, and what does compaction *not* improve?
- RQ5.** Can group-level paired evidence select a risk-bounded optimization action before a fresh real-record cohort, and does that choice preserve the registered task contract while reducing resource use?
- RQ6.** Which public agent benchmarks actually expose the state, effects, observed values, and independent groups needed to evaluate post-trace compilation, and what does an all-source executable integration reveal when those fields are absent?

The paper makes four contributions:

1. An evidence-licensed trace-to-program formulation that combines typed value provenance, effect and position barriers, bounded readable synthesis, empirical contracts, continuation-pinned composite projection, and runtime fallback.
2. A compile-or-retire admission protocol: the score and threshold grid are frozen before calibration, exact finite-sample bounds govern dispatch, and a framework-neutral portfolio can compare only transformations with paired evidence and compatible manifests.
3. A real-provider study on public records that separates execution conformance from source-grounded answer quality, exposes a depth-sensitive factual failure, and compares unchanged, compiled, provider-visible macro, fair pre-model manual, and bounded GEPA conditions without claiming superiority where the methods tie.
4. A revision-pinned ten-benchmark evidence map with a shared typed reference-plan IR. It executes five external paths, identifies only two trace-complete compiler substrates, and reports their refusal outcomes instead of averaging incompatible evidence.

We do *not* claim semantic equivalence, production certification, cross-domain generalization, or superiority to hand-written code. GCS synthesizes only a read-only view over an admitted program; it does not infer undeclared semantics or fuse physical reads. The six-case manual result is parity, and the bounded GEPA result does not generalize to prompt optimization as a whole.

Roadmap. Section 2 formalizes episodes, candidate regions, the position invariant, and the selective objective. Section 3 presents the method as a cascade of independent rejection points (fig. 1) and states the calibration guarantee. Section 4 defines the evidence tiers and the hypotheses they test, and section 5 answers RQ1–RQ6. Section 6 positions GAC against adjacent optimizers, and section 8 states what the evidence does not cover. Every rejected candidate, the archived failed pilot, and the unsupported claims are retained rather than pruned, so the denominator of each result stays visible.

2 Problem Formulation

2.1 Executions and candidate regions

An episode $E = (z, M, e_{1:T}, y)$ contains an entry-state snapshot z , a content-addressed execution manifest M , ordered events e_t , and an observable outcome y . Events include model requests/responses, tool calls/results, handoffs, guardrails, approvals, errors, and commit boundaries. Each tool f has an application-owned effect declaration $\epsilon(f)$ and capabilities such as *speculatable* and *replayable*. Unknown effects are not reads.

A candidate region $R = [a, b]$ begins at a model-request boundary and ends after a tool result. It is *groundable* when every call argument has an expression over entry state or prior results within R :

$$\forall c_j \in R, \forall u \in \text{args}(c_j), \quad u = g_{j,u}(z, o_{<j}), \quad g_{j,u} \in \mathcal{L}, \quad (1)$$

where $\mathcal{L}$ is a closed, bounded transform library. It is *effect-admissible* when all tools are declared read-only, speculatable, replayable, and within the same principal/isolation partition. Handoffs, approvals, writes, errors, and unknown effects are barriers.

The current runtime resolves artifacts at the initial model boundary. Therefore the deployable compiler adds a position invariant:

$$a = 0 \quad \text{in the normalized model-boundary index.} \quad (2)$$

This restriction is not aesthetic. A suffix program dispatched at entry may reorder or duplicate earlier calls; section 5.9 demonstrates the failure empirically.

2.2 Selective optimization objective

The compiler emits an artifact $A = (P, H, V, q, \eta, M)$: deterministic program P , hard guard H , verifier V , nonconformity score q , threshold η , and manifest pins M . For a future episode, dispatch is selective:

$$d_A(z) = \mathbb{I}\{H(z) = 1 \wedge q(z) \leq \eta \wedge M \text{ matches}\}. \quad (3)$$

If any precondition fails, the baseline agent runs. If execution fails before a commit and staging proves the attempt clean, the baseline agent runs. Dirty failure is an incident, not fallback.

Let $L(A, z) \in \{0, 1\}$ indicate a wrong compiled execution after dispatch and $C(A, z)$ its execution cost. We seek an artifact with high coverage and savings subject to bounded selective risk:

$$\max_A \quad \mathbb{E}[d_A(z)\{C(B, z) - C(A, z)\}] \quad (4)$$

$$\text{s.t.} \quad \Pr[L(A, z) = 1 \mid d_A(z) = 1] \leq \alpha, \quad (5)$$

where B is the unchanged agent. The implementation may return no artifact (RETIRE); this is a valid optimizer output.

At the compiler layer the implemented action set is $\{B, A\}$: retain the baseline or admit one compiled artifact. The separate portfolio layer in section 3.6 consumes paired measurements for arbitrary named actions. It does not weaken compiler admission or turn an unevaluated macro into deployable code.

Equation (5) is a design target, not a problem the implementation solves. Algorithm 1 ranks families by (6) and returns the *first* family that survives synthesis, contracts, and calibration; it neither enumerates all admissible families nor argues a bound on the gap to the best one. This matters more than a presentational note, because it makes the ranking load-bearing after all: since the first admissible family ships, the two coarse terms in (6) can change *which* artifact is registered even though neither participates in any admission decision. Reading ρ_{eff} from the effect catalog and evaluating all admissible families before selecting are both small changes, and both are future work.

3 Guarded Agentic Compaction

3.1 Typed provenance and canonical families

A capture adapter normalizes framework traces into the Episode IR. The OpenAI Agents SDK is a natural substrate because its agent loop records model calls, tools, agents, guardrails, and handoffs [34]; the compiler does not depend on the SDK. The artifact persists normalized episodes as canonical local JSONL with atomic snapshot replacement and strict validation. We intentionally removed an unused MLflow adapter: none of the experiments consumed its search or tracking surface, while retaining it added a second compatibility and privacy boundary. This simplifies reproduction but

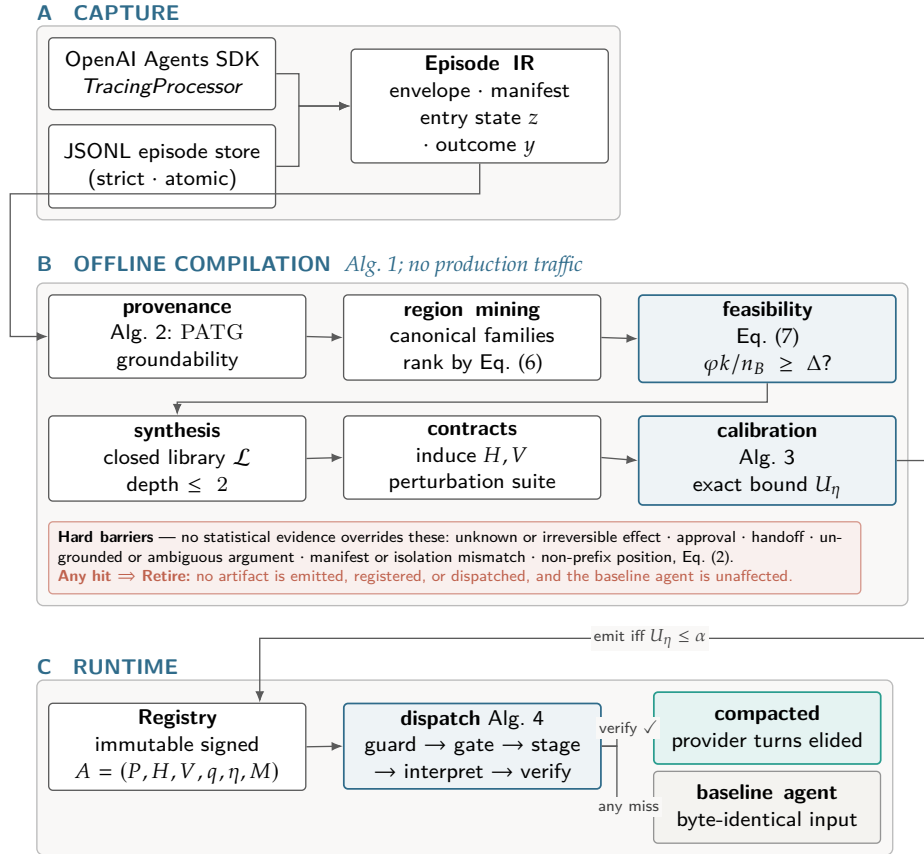


Figure 1. System architecture of GAC. **(A)** Capture normalizes framework traces into one typed Episode IR. **(B)** Offline compilation touches no production traffic and is a cascade of independent rejection points; the shaded *barrier* band lists conditions under which no statistical evidence can license an artifact. **(C)** Runtime resolves at the entry boundary. Of the terminal runtime edges, exactly one produces a compacted execution; five return the baseline and one raises an incident. “Baseline” is exact only where a staging owner holds the commit boundary: the outer runner can restore byte-identical model input, whereas the model-boundary adapter cannot un-emit a response the host has already committed to history (section 3.7.1).

gives up server-side trace search and leaves framework neutrality demonstrated by the typed seam and dependency layering, not by two independently validated foreign-trace mappings. Manifests pin prompt, policy, guardrail, tools, model, SDK, tracer, entry contract, and effect-catalog identities. Episodes with different compatibility keys are partitioned before learning.

Above that seam, an optimization-pass API composes GRC and trace-guided workflow specialization without coupling either algorithm to the SDK. The evidence portfolio is a later decision layer: it consumes paired measurements for named actions and cannot synthesize, approve, or execute an unmeasured macro. This separation keeps trace capture, program construction, empirical action selection, and runtime lifecycle as independently auditable boundaries.

For each call-argument slot, the program-argument trace graph (PATG) searches entry-state and prior-result paths for typed values and bounded transforms. Exact matches, stable field paths, and transformations in $\mathcal{L}$ become candidate producer edges. Ambiguous candidate sets above a configured cap and ungrounded values block a region. This is execution provenance in the database sense [10], specialized to tool arguments rather than tuple derivations.

The miner enumerates model-boundary-to-result windows with at most $w_{\max}$ tool calls, qualifies effects and live-ins, and hashes the tool signatures, dependency topology, run shape, and live-in/out shape while abstracting literal values. Families require support across independent scenario groups rather than raw event frequency; the implementation also supports a minimum-distinct-days requirement, which the live study of section 5.3 does not exercise (it sets `min_days` = 1 over a single-snapshot corpus). With N episodes, T events, and a bounded call window $w_{\max}$, enumeration is practically bounded by the number of reachable result boundaries; the implementation’s nested scan is worst-case $O(NT^2)$ when non-call spans are adversarial. We report this rather than the stronger $O(NT)$ claim sometimes suggested by the bounded-call intuition.

Surviving families are ranked by a minimum-description-length trade-off that pays for expected savings and charges for variance, effect exposure, and program size:

$$\text{score}(F) = \underbrace{s_F \bar{k}_F c_m}_{\text{expected saving}} - \lambda_1 H(F) - \lambda_2 \rho_{\text{eff}}(F) - \lambda_3 |F|, \quad (6)$$

where s_F is scenario-group support, $\bar{k}_F$ the mean removable model requests, c_m their unit cost, and $H(F)$ the Shannon entropy over canonical variants inside the family. The penalties matter: without λ_1 the ranking prefers a heterogeneous family whose single canonical form fits none of its members.

Two terms are coarser in the implementation than the notation suggests, and we state them as implemented. ρ_{eff} is intended as the declared-effect exposure of the region, but the shipped ranking approximates it by the fraction of tool names containing a namespace separator — a naming convention, not a reading of the effect catalog. $|F|$ is intended as the size of the program the family would produce, but ranking precedes synthesis, so the implementation substitutes the argument-slot count of the first observed window. Both appear only in a *ranking* heuristic: no admission decision depends on either, and the effect catalog is consulted where it is load-bearing — in qualification and in the runtime facade. Making them faithful would reorder candidates and therefore require re-running the offline study; we list it as future work rather than describe semantics the code does not enforce.

Before any synthesis runs, a feasibility estimator bounds what the compiler could achieve even with a perfect verifier and a gate that never abstains:

$$\Delta_{\max} = \frac{\varphi k}{n_B}, \quad \text{necessary: } \varphi \rho k \geq \Delta n_B, \quad (7)$$

with n_B baseline model requests per episode, φ the fraction of episodes holding at least one eligible region, k the requests removed per successful dispatch, and ρ the verifier pass rate. Because (7) assumes $\rho = 1$, no measured reduction can *exceed* it, and it is met with equality exactly when nothing abstains and nothing fails. That is what happens in section 5.3: with $\varphi = 1$, $k = 3$ and $n_B = 4$ the ceiling is 0.750 and the measured reduction is 75.0%. A saturated ceiling carries no information about the compiler — it confirms only that the task fully determined the region, which for that workload it does. Reporting the ceiling first makes a decisive negative answer cheap: a workload whose ceiling is under the target can be declined without building a compiler for it.

3.2 Bounded synthesis and contracts

Program synthesis searches a 23-operator library with depth at most two. Operators cover path selection, indexing, string normalization, arithmetic, comparisons, bounded

collection operations, and narrow loops. Candidate bindings are ranked by stability across groups; a group-wise refit rejects bindings that exploit row-level coincidence. The approach resembles programming-by-example [17] and bounded sketching [39], but the hypothesis space is deliberately small enough to inspect. It does not generate arbitrary Python.

Hard guards constrain entry types, required fields, categorical sets, numeric intervals, patterns, isolation keys, allowed effects, and manifest pins. Verifiers constrain live-out presence, type, cardinality, empirical hulls, provenance, effect multisets, and call counts. These are likely invariants [15], not proofs for all future inputs. Replay and perturbation are therefore available to challenge a candidate before calibration, but they are only as strong as the substrate supplied: without a sandbox the suite cannot run at all, and the artifact then records `perturbations_claimed: false` rather than implying a challenge occurred. The headline live artifact of section 5.3 is in exactly that position, and we report it there as a limitation of that run rather than as a property of the method.

3.3 Guarded composite synthesis

The macro comparison revealed a structural asymmetry: the manual baseline may redesign the application interface, while the original compiler preserves every source-tool observation. GCS closes that specific gap without moving application code into the optimizer’s trust boundary. After ordinary synthesis and admission, it packages the same program as $\tilde{P} = (P, \Pi, \tau_c)$, where P is the verified internal program, Π is a declared projection over verified live-outs, and τ_c is the exact compatibility key of the continuation that consumes the projection. A tool must carry an explicit batchable capability before it can appear inside this interface.

Some recurrent calls differ only in arguments that are irrelevant to the registered task. The effect catalog may therefore attach a signed, declarative task-semantic canonicalization to a dotted argument path. The implementation admits only five closed operations (integer clamping, whitespace stripping, case folding, stable set-like sorting, and finite aliases), and integer rules may declare an admissible input domain. This is not learned equivalence: values outside that domain raise and deoptimize. In the GitHub task, the consumer uses at most the first three comments and the snapshot tool already caps its result at three, so observed limits of at least three share the representative `limit=3`; `limit=1` is deliberately outside the contract.

The projection language reuses the bounded binding DSL. It can select only existing verified live-outs and cannot call arbitrary code, resolve a dynamic tool, or access unverified state. Runtime execution retains all internal source calls, effects, and field-level provenance. Verification and projection both complete inside the staging boundary; a missing projected field returns the baseline before release. A pre-model dispatch additionally requires an exact continuation-manifest match. It then exposes one native composite observation to the provider, so the provider receives the sufficient record on its first request rather than spending a request selecting the composite.

GCS is consequently narrower than automatic API fusion. It does not parallelize the internal calls, reduce the number of source reads, generate a new remote service, or prove the application-supplied task-semantic contract. Its contribution is a serializable, fail-closed interface transformation over a program the existing compiler can already verify.

Algorithm 1 The end-to-end pipeline an operator runs, from qualified traces to a registered artifact. Every **retire** is recorded with its stage so rejections stay in the denominator. Stages marked (CALLER) are separate library entry points rather than steps inside `compile_grc()`, which receives the split object and does not invoke the feasibility estimator itself; the cascade is presented in one place because that is the order in which the stages must run, not because one function performs all of them.

Input: episodes $\mathcal{E}$; effect catalog C ; entry schema Θ ; risk budget α ; confidence budget δ ; grid Λ

Output: artifact $A = (P, H, V, q, \eta, M)$, or RETIRE with an attributed reason

```

1:  $\mathcal{E} \leftarrow \text{QUALIFY}(\mathcal{E}, C)$  ▷ drop partial runs, unpinned manifests, missing payloads
2:  $\{\mathcal{E}_M\} \leftarrow \text{PARTITIONBY}(\mathcal{E}, \text{manifest}, \text{principal}, \text{isolation})$ 
3: for all partitions  $\mathcal{E}_M$  do
4:    $(\mathcal{T}, \mathcal{D}, \mathcal{K}, \mathcal{S}) \leftarrow \text{GROUPEDSPLIT}(\mathcal{E}_M)$  (CALLER) ▷ train / dev / calibration / sealed test, split by group
5:    $G \leftarrow \text{BUILDPATG}(\mathcal{T}, \Theta)$  ▷ Alg. 2
6:    $\mathcal{W} \leftarrow \text{MINEREGIONS}(G, C, w_{\max})$ 
7:    $\mathcal{F} \leftarrow \text{CANONICALIZE}(\mathcal{W})$  ▷ hash signature, topology, run and live-in/out shape
8:   keep  $F \in \mathcal{F}$  with cross-group support (and cross-day, if configured)
9:   if  $\varphi k/n_B < \Delta$  then (CALLER)
10:    return RETIRE (infeasible ceiling) ▷ a separate estimator pass; run it before paying for synthesis
11:   end if
12:   for all families  $F$  ranked by score (6) do
13:      $P \leftarrow \text{SYNTHESIZE}(F, \mathcal{L})$  ▷ closed library, depth  $\leq 2$ , group-wise refit
14:     if  $P = \perp$  then
15:       continue (ungroundable slot)
16:     end if
17:      $(H, V) \leftarrow \text{INDUCECONTRACT}(F)$ 
18:     if  $\neg \text{REPLAYANDCHALLENGE}(P, V, \mathcal{D})$  then ▷ requires a sandbox; records perturbations_claimed otherwise
19:       continue (counterexample) ▷ stale reads, reordering, empty sets, drift, faults
20:     end if
21:      $q \leftarrow \text{FITSCORE}(\mathcal{D})$ ; freeze  $q$  ▷ dev groups only; never calibration
22:      $\eta \leftarrow \text{CALIBRATE}(q, \mathcal{K}, \Lambda, \alpha, \delta)$  ▷ Alg. 3
23:     if  $\eta = \perp$  then
24:       continue (no admissible threshold)
25:     end if
26:     return PACKAGE( $P, H, V, q, \eta, M$ ) with evidence and lifecycle
27:   end for
28: end for
29: return RETIRE

```

3.4 Fair manual and learned-optimizer interfaces

To separate runtime placement from automatic discovery, we implement a second pre-model runner that does not consume a compiler artifact or its calibrated gate. It accepts an explicit, hand-authored read program, source and continuation manifests, effect catalog, bounded projection, and output/provenance/call-count verifier. Manifest drift, an undeclared effect, a missing field, a repeated observation, or any verifier failure returns the unchanged agent before release. This is a guarded laboratory baseline, not an independently reviewed production program; its construction cost is not measured.

For learned prompt optimization we wrap the official GEPA `optimize_anything` interface behind a provider-agnostic adapter. The adapter requires disjoint train and

Algorithm 2 BUILDPATG — typed argument provenance. A slot with no witness is a genuine model decision; a slot with too many witnesses is ambiguous. Both block the region rather than defaulting to the cheapest explanation.

Input: episodes $\mathcal{T}$; entry schema Θ ; library $\mathcal{L}$; ambiguity cap κ
Output: provenance graph G with an edge per grounded argument slot

```

1:  $G \leftarrow \emptyset$ 
2: for all episodes  $E = (z, M, e_{1:T}, y) \in \mathcal{T}$  do
3:    $\Sigma \leftarrow \{("z", \pi, v) : (\pi, v) \in \text{FLATTEN}(z), \pi \in \Theta\}$  ▶ only allowed entry paths are eligible
4:   for all tool calls  $c_j \in e_{1:T}$  in order do
5:     for all argument slots  $u \in \text{args}(c_j)$  do
6:        $\mathcal{H} \leftarrow \{\sigma \in \Sigma : \exists g \in \mathcal{L}, g(\sigma) = u, \text{depth}(g) \leq 2\}$ 
7:       if  $u$  is declared literal-only for  $c_j$  then
8:          $\mathcal{H} \leftarrow \{\sigma \in \mathcal{H} : g = \text{id or } g \text{ constant}\}$  ▶ never reconstruct a page index or a limit
9:       end if
10:      if  $\mathcal{H} = \emptyset$  then
11:        mark  $c_j$  MODEL-ORIGINATED; block region
12:      else if  $|\mathcal{H}| > \kappa$  then
13:        mark  $c_j$  AMBIGUOUS; block region
14:      else
15:         $G \leftarrow G \cup \{\sigma \xrightarrow{g} (c_j, u)\}$  for the minimum-description-length  $\sigma$ 
16:      end if
17:    end for
18:     $\Sigma \leftarrow \Sigma \cup \{(\text{var}(c_j), \pi, v) : (\pi, v) \in \text{FLATTEN}(o_j)\}$  ▶ the result becomes a witness for later slots only
19:  end for
20:  for all pairs  $(e_s, e_t)$  sharing a resource where either writes it do
21:     $G \leftarrow G \cup \{e_s < e_t\}$  ▶ order edge: a reordering can never be proposed across a conflict
22:  end for
23: end for
24: return  $G$ 

```

validation identifiers, enforces hard proposal, metric-call, and candidate-length budgets, records a sanitized audit log, and leaves tracking disabled. Only one operational-strategy sentence is mutable; the factual output schema, safety instructions, tools, grader, and quality-dominant objective remain fixed. Prompt optimization and guarded program execution therefore change orthogonal variables and can be evaluated alone or together. Optimization requests, tokens, latency, and cost are accounted separately from held-out deployment.

3.5 Exact risk-gated admission

The score $q(z)$ is a logistic model over entry-observable features: unseen category, distance proxy, missing fields, hull margin, temporal drift, provenance ambiguity, and branch entropy. It is trained on development-group *unproductive* outcomes (wrong or abstained), then frozen. Safety calibration uses only *violations* (wrong after dispatch). The fixed threshold grid is

$$\Lambda = \{.02, .05, .08, .11, .14, .17, .20, .25, .30, .40, .50\}.$$

For threshold η , let n_η calibration groups be admitted and k_η contain at least one

violation. The compiler computes a one-sided Clopper–Pearson upper bound [13]

$$U_\eta = \text{Beta}^{-1}\left(1 - \frac{\delta}{|\Lambda|}; k_\eta + 1, n_\eta - k_\eta\right), \quad (8)$$

with the $k = 0$ closed form $U_\eta = 1 - (\delta/|\Lambda|)^{1/n_\eta}$. The largest-coverage admissible threshold with $U_\eta \leq \alpha$ is selected; if none exists, the family retires.

Proposition 1 (Simultaneous calibration statement). *Assume the candidate program, score, violation definition, and finite grid Λ are fixed before calibration, and that admitted calibration groups contribute independent and identically distributed Bernoulli violation indicators drawn from the same distribution as future admitted groups (or are conditionally i.i.d. given the conditioning variables used to define the sampling unit). Then, with probability at least $1 - \delta$ over the calibration sample, every grid threshold’s true group-level selective violation rate is no greater than its computed U_η . Consequently, any selected threshold satisfying $U_\eta \leq \alpha$ has violation rate at most α on the same distribution.*

Two words in the proposition carry weight. A *violation* is disagreement with V , and V is *induced* from the same training traces the program was synthesized from, so the guarantee bounds the rate at which a compiled region breaks its own registered contract — not the rate at which the agent’s task outcome is wrong. Absent distribution shift a program fitted to satisfy V will rarely violate V , which is why zero violations in 92 groups should be read as self-consistency rather than as correctness. Three things would make the chain independent, and none is present in the shipped artifact: a perturbation challenge (not run — `perturbations_claimed: false`), an adjudication oracle separate from the induction traces, and a task-quality measure that can detect fabrication (section 8). Algorithm 3’s α should therefore always be read as bounding *contract violation*, never as bounding error.

The statement follows from one-sided exact-binomial coverage at $1 - \delta/|\Lambda|$ and the union bound. The independence assumption is load-bearing and stronger than exchangeability, which permits dependence that exact binomial coverage does not tolerate: groups drawn from one repository, tenant, time window, or workflow family are plausibly correlated, and the scenario-group unit is what must carry independence. A clustered deployment should calibrate at the cluster level and check sensitivity with a block bootstrap or a beta-binomial model. The union bound also covers only the grid for *one* frozen artifact; a portfolio of artifacts, or repeated promotion of the same one, needs an explicit risk allocation across those decisions. It is a calibration guarantee, not proof under drift or adversarial dependence; this is consistent with learn-then-test risk control [3, 7]. With $\alpha = .05$, $\delta = .10$, $|\Lambda| = 11$, and zero violations, at least 92 admitted group records are needed; the guarantee additionally requires those records to satisfy the independence assumption above.

That number is a consequence of our parameters, not a discovery about any workload, and it is worth stating how sensitive it is. Holding $\alpha = .05$ and $\delta = .10$ fixed but pre-committing to a *single* threshold rather than a grid of eleven gives $n \geq \log \delta / \log(1 - \alpha) = 45$; relaxing α moves it further. So when we report that no NESTFUL family clears the requirement, the honest reading is that none clears *this* configuration — a grid-searched threshold at 5% risk with 90% confidence. Roughly half of the families would still fail at $|\Lambda| = 1$, since the largest support is 26, but the gap between 26 and the requirement is a design parameter as much as a data property.

Algorithm 3 CALIBRATE — fixed-grid exact selective admission. The grid and the score are frozen before this procedure sees a calibration group, which is what makes the union bound legitimate.

Input: frozen score q ; calibration groups $\mathcal{K}$; grid Λ ($|\Lambda| = 11$); risk budget α ; confidence budget δ

Output: threshold η with simultaneous guarantee, or $\perp$ (RETIRE)

```

1:  $\mathcal{A} \leftarrow \emptyset$  ▷ admissible (coverage, threshold) pairs
2: for all  $\eta \in \Lambda$  do
3:    $K_\eta \leftarrow \{g \in \mathcal{K} : \exists z \in g, q(z) \leq \eta \wedge H(z) = 1\}$  ▷ groups this threshold would admit
4:    $n_\eta \leftarrow |K_\eta|$ 
5:    $k_\eta \leftarrow |\{g \in K_\eta : g \text{ contains a violation}\}|$  ▷ wrong after dispatch only, never abstention
6:   if  $n_\eta = 0$  then
7:     continue
8:   end if
9:    $U_\eta \leftarrow \text{Beta}^{-1}\left(1 - \frac{\delta}{|\Lambda|}; k_\eta + 1, n_\eta - k_\eta\right)$  ▷ Eq. (8); closed form  $1 - (\delta/|\Lambda|)^{1/n_\eta}$  when  $k_\eta = 0$ 
10:  if  $U_\eta \leq \alpha$  then
11:     $\mathcal{A} \leftarrow \mathcal{A} \cup \{(n_\eta/|\mathcal{K}|, \eta)\}$ 
12:  end if
13: end for
14: if  $\mathcal{A} = \emptyset$  then
15:   return  $\perp$  ▷ with  $k = 0$  this needs  $n_\eta \geq \log(\delta/|\Lambda|)/\log(1 - \alpha) \approx 92$ 
16: end if
17: return  $\arg \max_{(\text{cov}, \eta) \in \mathcal{A}} \text{cov}$  ▷ largest coverage among certified thresholds

```

3.6 Risk-bounded transformation portfolio

The compiler’s admission question and the application’s engineering choice are different. After candidate actions have been measured on the same independent workflow groups, the portfolio layer compares $\mathcal{A} = \{B, K, C, M, \dots\}$: unchanged baseline, cache, compiler, reviewed macro, or an application-defined action. Missing evidence creates no candidate row and therefore cannot be imputed into a recommendation.

For group g and action a , let $r_{g,j}(a) = 1 - C_{g,j}(a)/C_{g,j}(B)$ be the paired reduction for metric j , and define

$$U_g(a) = \sum_j w_j r_{g,j}(a), \quad \sum_j w_j = 1, \quad w_j \geq 0. \quad (9)$$

The pilot assigns weights 0.35, 0.25, 0.20, and 0.20 to cost, latency, tokens, and tool calls, respectively. It records a quality violation whenever the candidate’s task contract fails and a regret event whenever $U_g(a) \leq U_{\min}$. Repeated executions of one issue are averaged within that issue; they never increase the independent-group count. For every observed non-baseline action, we compute one-sided Clopper–Pearson bounds for quality and regret, denoted $R_{\text{quality}}^+(a)$ and $R_{\text{regret}}^+(a)$. The confidence error budget is split across both bounds and all compared actions. The admitted set is

$$\mathcal{A}_{\text{ok}} = \{a : R_{\text{quality}}^+(a) \leq \alpha_q, \quad R_{\text{regret}}^+(a) \leq \alpha_u, \quad n_a \geq n_{\min}, \quad M_a = M_*\}. \quad (10)$$

The selector returns $\arg \max_{a \in \mathcal{A}_{\text{ok}}} \bar{U}(a)$, or B if the set is empty. Manifest mismatch invalidates the decision at runtime. Macros default to *human review*; the library returns a decision record, evidence, and rationale, not generated application code. Exact bounds cover the registered binary endpoints under the same group assumptions as compiler

Algorithm 4 DISPATCH — boundary-time admission. Cheap deterministic checks reject first and the calibrated gate runs only on what survives them. Exactly one exit path compacts; the rest return the unmodified agent, and only a dirty post-commit failure raises an incident.

Input: entry state z ; runtime context M' ; registry $\mathcal{R}$; mode $m \in \{\text{off}, \text{shadow}, \text{live}\}$

Output: observations for the region, or FALLBACK to the baseline agent B

```

1: if  $m = \text{off}$  then
2:   return FALLBACK ▷ conformance: model-visible input must be byte-identical
3: end if
4:  $A \leftarrow \mathcal{R}.\text{RESOLVE}(M'.\text{compatibility}, M'.\text{partition})$  ▷ constant time
5: if  $A = \perp \vee A.\text{lifecycle} \neq \text{ACTIVE} \vee \neg \text{VERIFYSIGNATURE}(A)$  then
6:   return FALLBACK
7: end if
8: if  $H_A(z, M') \neq 1$  then
9:   return FALLBACK ▷ manifest pins, isolation keys, typed hulls, effect set
10: end if
11: if  $\text{tools}(A.P) \cap \text{ALREADYOBSERVED} \neq \emptyset$  then
12:   return FALLBACK ▷ the region already ran; its live-ins are not the fitted ones
13: end if
14: if  $m = \text{live} \wedge \exists f \in \text{tools}(A.P) : C(f).\text{quota\_attested} \wedge \text{SNAPSHOT} = \perp$  then
15:   return FALLBACK ▷ no staging owner, so a discarded attempt is not attestable
16: end if
17: if  $q_A(z) > \eta_A$  then
18:   return FALLBACK ▷ calibrated abstention on an uncertain input
19: end if
20: if  $m = \text{shadow}$  then
21:   log would-dispatch; return FALLBACK ▷ observe coverage without changing behaviour
22: end if
23:  $S \leftarrow \text{STAGE.BEGIN}()$ 
24:  $r \leftarrow \text{INTERPRET}(A.P, z, \text{FACADE}(C))$  ▷ bounded program; the facade re-checks effects at execution time
25: if  $r$  raised PRECOMMITERROR then
26:   assert  $S.\text{REVERSIBLE}()$ ; return FALLBACK
27: else if  $r$  raised POSTCOMMITERROR then
28:   return INCIDENT ▷ an external commitment happened; do not feign rollback
29: end if
30: if  $V_A(r) \neq 1$  then
31:   discard  $r$ ; return FALLBACK ▷ live-outs, effect multiset and call count are checked before use
32: end if
33:  $S.\text{COMMIT}(r.\text{effects})$ ; return  $r.\text{observations}$ 

```

calibration. They do not make the weighted utility canonical, price engineering effort, or prove optimality over an unmeasured action.

3.7 Runtime and framework integration

At runtime, resolution is a bounded lookup: the registry indexes artifacts by compatibility and partition key, then filters the small candidate list at that key by lifecycle, kind, and signature. Cost is therefore constant in registry size but linear in the number of artifacts sharing one key, which the packaging step keeps small rather than the data structure guaranteeing it. The dispatcher checks lifecycle, manifest, hard guard, calibrated gate, mode, budget, and quota snapshot; executes through a permission facade into a staging area; verifies the result; and commits only on success. Shadow mode records would-dispatch behavior but does not change the agent. Live resolution accepts only ac-

tive artifacts. The current version compiles reads only; irreversible operations remain with the baseline agent.

For the OpenAI Agents SDK, the library provides trace capture plus a custom model-boundary adapter for local function tools. Streaming, hosted tools, MCP tools, handoffs, loops, and assertions bypass rather than degrade. The SDK provides the loop and lifecycle, but it is not itself an offline optimizer [34].

3.7.1 Where “unmodified baseline” is exact, and where it is not

The two integration paths do not offer the same guarantee, and the difference is structural rather than an implementation gap. The outer runner owns the entry snapshot and the commit boundary, so a rejected attempt restores byte-identical model-visible input: for that path, “the baseline runs unchanged” is exact at every failure point. The model-boundary adapter cannot make the same promise. Once it has returned a synthesized `ModelResponse`, the host runner may already have committed that item to session history, and the `Model` interface provides no way to withdraw it; a verifier failure after that point delegates to the wrapped model with history the baseline would never have contained. Every check that can reject *before* emission — lifecycle, signature, manifest pins, hard guard, already-observed tools, quota attestation, and the calibrated gate — is therefore exact for both paths, and only post-emission verifier or binding failures are weaker for the adapter. Claims of universal clean fallback in this paper should be read as scoped to the staging-owning runner; the adapter is guarded substitution with explicitly weaker post-emission semantics. A general solution requires artifacts to target explicit control points with continuation tokens rather than implicit model boundaries, which we leave to future work (section 7.3).

4 Experimental Methodology

4.1 Evidence tiers and hypotheses

We use three non-overlapping evidence tiers, ordered by what each can license.

Tier 1 — external benchmark interoperability and compiler evidence. A sealed manifest pins NESTFUL plus nine complementary benchmark families. All ten receive an explicit adapter, execution, or gate disposition; unavailable prerequisites never become zero scores. NESTFUL and API-Bank are the only two paths with complete observed values that can exercise the compiler. BFCL executes its official checker, Tool-Sandbox, τ^2/τ^3 , and BrowseComp receive bounded official live-provider runs, and the remaining sources receive adapter/preflight evidence. These substrates test different objects, so we report a ledger rather than pool them into one score.

Tier 2 — real records, live provider. Six GitHub protocols use actual public issue records, deterministic local reads over a pinned snapshot, and live OpenAI provider calls. The first prescribes a fixed prefix as a conformance ablation. The review-driven follow-up names required facts but neither tool functions nor order, grades exact source facts independently of the trace, and adds a live composite-tool baseline. An expanded replication seals 132 discovery and 30 balanced test records, exercises compiler abstention from an ungroundable third call, and repeats all three conditions. A prospective portfolio pilot freezes the replication as selection evidence, chooses one reviewed action, and runs only it and the unchanged agent on 12 fresh records. Finally, an exploratory GCS study compiles the full three-read region from the retained discovery traces and compares its pre-model composite with the provider-visible hand-written macro on 12 further records excluded from every prior cohort. They are evidence about agent con-

trol flow on real records, not live GitHub service reliability. A final bounded comparator study excludes every earlier issue identifier, freezes disjoint 4/2/6 optimization-train/validation/test splits before any provider outcome, and runs unchanged, GEPA-only, GCS, GCS+GEPA, and independently authored pre-model conditions on the same six held-out records. The snapshot contains real records and every condition makes live provider calls; no response is simulated.

Tier 3 — controlled demonstration suite. Six workflow *shapes* are run end to end through the real SDK runtime with live provider calls against deterministic local services. The business records here are fictional, so this tier cannot support a claim about real-world data; what it can do is cover control-flow structures the other two tiers do not contain at all — observation-dependent branches, pagination, a mandatory irreversible write, a handoff barrier, an undeclared-effect tool surface, and route specialization — and expose the behaviour of the runtime when it must *refuse*. We report it because a suite in which every case succeeds would be evidence of a weak test set, not a strong compiler. Three of its eight conditions are negative controls whose only correct outcome is no compaction.

The directional hypotheses below were fixed internally before the sealed test was scored, but the study was not externally preregistered: (H1) provenance places the expected producer in the candidate set for more than 90% of executable nested dependency slots — a recall statement, reported alongside unique-resolution and precision rates so it cannot be read as exact reconstruction; (H2) the compiled condition reduces provider requests and total tokens; (H3) no observed quality loss occurs on the sealed task; (H4) families below the exact-gate sample requirement retire; and (H5) a condition that must refuse reproduces the baseline model-call count and quality. H5 is deliberately *not* stated in dollars: section 5.4 shows refusing conditions costing more than baseline, for prompt-cache reasons unrelated to whether the refusal was correct. H3 is an observed-sample hypothesis, not a population equivalence claim. The portfolio pilot was designed after the replication was known but before its fresh cohort was selected or executed. Its prospective hypothesis (H6) is therefore narrower: the frozen selector’s chosen action will pass all observed fresh task contracts and reduce its weighted resource objective relative to baseline. H6 was not externally preregistered and cannot establish that selection beats an always-macro policy. GCS was designed after both macro results were known. Its 12-pair study is therefore an explicitly post-study, exploratory test rather than an additional preregistered hypothesis. The issue selection is provider-outcome-free and disjoint from all 424 earlier issue IDs, but it was recorded by the same run that executed the provider calls rather than sealed in an externally timestamped protocol. The learned/manual comparator was likewise designed after the GCS result and is explicitly exploratory. Its test identifiers were frozen before prompt optimization, but the study was not externally preregistered and its six cases are not a powered non-inferiority test.

4.2 External benchmark: NESTFUL

NESTFUL contains more than 1,800 executable nested API sequences and was designed to test whether later calls correctly reuse earlier outputs [6]. We pin upstream commit `f2c4123e735` (the full identifier is in the source manifest) and verify SHA-256 digests before use. Of 1,861 records, 1,416 use the audited `basic_functions` module; 1,415 execute successfully. One upstream record raises a string/integer type error and remains in the failure log.

We convert each successful execution into the same typed Episode IR, reconstruct expected producer references, mine families, split within families by group, synthesize from train/dev windows, and replay on held-out windows. This experiment does not call an LLM and does not measure NESTFUL planning accuracy. It tests the compiler after a valid trace exists. The exact gate is then applied to observed family support.

4.3 All-source benchmark interoperability audit

We additionally pin BFCL v4, API-Bank, ToolSandbox, the maintained τ^2/τ^3 implementation, ToolBench, AgentBench, GAIA, SWE-bench Verified, and BrowseComp at exact Git revisions or dataset digests [5, 21, 26–28, 32, 37, 38, 44]. A framework-neutral ReferenceTask IR retains revision, substrate, group, ordered actions, conservative effects, and whether outputs were actually observed. It refuses conversion into an executable Episode when outputs are absent and forbids pooling revisions or substrates. The effect labels in this audit are screening annotations, not signed runtime declarations.

The same acquisition script records all prerequisites. Eight benchmark task sets are screened, covering 5,419 tasks and 17,836 reference actions; GAIA remains authorization gated, ToolBench is limited to ten versioned repository fixtures, AgentBench requires its external services and Freebase bytes, and SWE-bench execution is gated because the available Docker host is arm64 with 12.5 GiB rather than the harness’s recommended x86-64 and 16 GiB. We do not impute scores for those paths.

For bounded official executions, BFCL validates 200/200 pinned gold plans without claiming model accuracy. One ToolSandbox scenario uses real provider calls and obtains 0.982 milestone similarity. Four τ domain tasks use real provider calls and obtain 0/4 end-to-end reward while consuming 288,757 tokens. Three sealed BrowseComp tasks use hosted web search and obtain 1/3 accuracy with 28 searches and 237,859 tokens. Hosted search lacks a replay-safe local tool-result contract, so it is a required compiler bypass. ToolSandbox and τ are public simulated environments, not real-world demos; BrowseComp is a live-web benchmark subset, not a paired compiler experiment or official leaderboard submission.

4.4 Controlled fixed-prefix live study

We pin revision e344be7b84d1 (full identifier in the source manifest) of the Apache-2.0 helmo/github-issues dataset [19], verify the 12.7-MB Parquet digest, exclude pull requests, require at least 80 body characters, and deduplicate issue numbers. The scenario asks an agent to classify a huggingface/datasets issue as bug, enhancement, or question and return a short evidence-grounded summary. Three local function tools read the exact issue record, labels, and first three comments from the frozen public snapshot. The records and workload are real; the local tool service is deterministic so that API and website drift cannot contaminate paired comparisons.

Every model turn uses the OpenAI Agents SDK 0.19.2, OpenAI Python 2.52.0, and gpt-5.6-luna at low reasoning effort. The script loaded OPENAI_API_KEY for provider calls and HF_TOKEN for dataset retrieval; it serialized only boolean usage flags, never credentials. Cost is an estimate from the model-price table frozen with the experiment and the official pricing source [35].

The discovery cohort contains 132 issues, and all 132 satisfy the exact tool contract, so all 132 are compiler-eligible. The split consumes the first 116 under a fixed rule — train (16), development (8), calibration (92), sizes chosen before any outcome was scored, because $n = 92$ is fixed a priori by $(\alpha, \delta, |\Lambda|)$ — and leaves the remaining 16 unused. Those

16 are surplus to the calibration requirement, not filtered: they are neither excluded for a quality reason nor drawn upon, and we state the residue rather than report a denominator of 116 as though eligibility had produced it. A stronger design would either consume them or size the cohort to the split. The compiler, unmodified for the task, emits the prefix

$$\text{record}(z.id) \rightarrow \text{labels}(\text{record}.id) \rightarrow \text{comments}(\text{record}.id, 3).$$

It blocks all 116 suffix candidates under the position invariant. The 18-item sealed test contains six issues per category, is disjoint from discovery, and excludes all 162 issues touched by the earlier pilot. Baseline and compiled conditions run the same task and tools. Six cases are repeated once for determinism analysis.

The baseline requires four provider responses around three tool calls. The compiled condition executes the same necessary tools natively and asks the model once to produce the final structured response. Thus the intervention tests decision elision, not tool elision or parallel scheduling.

4.5 Natural-order, source-grounded live studies

The first follow-up keeps the pinned dataset, model, SDK, and three read tools but changes the task contract. Its prompt requests exact issue number, title, state, label-derived category, evidence label, and a verbatim excerpt from one of the first three comments; it contains no tool function name and no execution order. The grader checks the structured facts against the snapshot and normalized excerpt containment against the returned source comments. Tool order is reported separately and contributes nothing to task quality. Executable regressions verify that reordering calls leaves the score unchanged and that a fabricated comment excerpt fails.

We exclude all 312 records touched by the fixed-prefix final run or pilot. A stable hash selects 80 fresh discovery issues and 18 disjoint test issues (5 bug, 2 enhancement, 11 other; no class quota). All 80 discovery agents independently choose $\text{record} \rightarrow \text{labels} \rightarrow \text{comments}$, and all pass the corrected source-grounded contract. The executed artifact was built under the online oracle, which falsely rejected one short but valid excerpt: 79 records were then eligible, a stable hash assigned 20 train, 10 development, and 45 calibration records, and five were unused. Under the corrected oracle, issue 1741 would rank sixth and replace issue 3511 in that split. We neither retrain the executed artifact post hoc nor assume the two artifacts are equivalent; the result manifest records the consequence. No filter uses tool order. The review-driven protocol was checked into executable tests before provider execution but was not externally preregistered.

The paired test adds a hand-written `issue_get_bundle` tool that returns the same record, labels, and three comments in one read. Each issue runs the unchanged agent, learned compiler, and macro exactly once. The six permutations of condition order are cycled three times across the 18 records, so every condition occupies every ordinal position equally often. All conditions use the same model and factual output schema. The full run made actual provider calls and records an estimated total cost of \$0.0920; credentials are represented only by boolean usage flags.

The expanded replication uses the same provider and snapshot but a stricter typed output: exact issue number, title, state, total comment count, label-derived category and evidence label, plus a 20–240 character verbatim excerpt from the returned title, body, or comment payload. A provider-free preflight sealed 132 discovery records and 30 held-

out records, balanced as ten bugs, ten enhancements, and ten questions. The paid run used those exact records and cycles all six baseline/compiler/macro orders five times. Ten test records are repeated once per condition for determinism.

Of the 132 discovery outputs, 130 pass the exact-source task and are eligible under the executed safety rule. All 132 read the record, then labels, then comments; however, the natural caller chooses different integer comment limits. Because no trace-grounded expression can reproduce that argument, the three-read candidate fails with `ungroundable_slot`. The compiler correctly falls back to the longest groundable prefix, `record→labels`, and the ordinary agent performs the comments read and final rendering. Sixteen training, eight development, and 92 calibration records are selected from the 130 eligible traces; the other 14 are unused. The complete run retains 252 agent outputs, 848 provider responses, no infrastructure failure, and an estimated total cost of \$0.1913. An online literal-argument grader was corrected provider-free: all original outputs and measurements remain bound to the paid discovery checkpoint, while 212 prior quality objects are preserved under `online_quality`.

4.6 Prospective portfolio pilot

The portfolio calibration consumes only the 30 independent primary issue groups from the expanded replication. Compiler and macro observations are paired with the unchanged arm; ten repeated executions are averaged within their issue and do not increase support. We fix $U_{\min} = 0$, $n_{\min} = 30$, $\alpha_q = \alpha_u = .15$, and 95% familywise confidence split over two actions and two exact bounds. This 15% pilot limit is intentionally looser than the compiler’s 5% registered-contract bound and must not be conflated with it.

Before any prospective call, the script stores the decision and hashes the calibration file. A stable hash then selects 12 previously unused public issues: four bugs, four enhancements, and four “other” records. All eligible question records had already been consumed by earlier sealed protocols, so reusing them would have violated the prospective boundary. Each issue runs baseline and the selected reviewed action once; order is counterbalanced six each way. The non-selected action is not executed on this cohort. The provider-backed script requires an explicit macro-review flag, records 24 unique native traces, and serializes no credential. This evaluates a pre-data action choice on a fresh cohort, but only within one workflow family and one model configuration.

4.7 Bounded learned and pre-model comparator study

We first reconstruct both GCS and the manual program on all 12 selected split records without a provider call. Their projected evidence must match byte-for-byte before the paid study may proceed. Four records train prompt proposals, two evaluate them, and six remain sealed for deployment; all are disjoint from 443 issue identifiers used by earlier experiments. Strict exclusion leaves bug, enhancement, and other records but no question record, so the held-out cohort balances only the three available categories (two each).

Official GEPA 0.1.4 may make at most 16 task-metric calls and three proposals. Each task evaluation is a real Agents SDK execution, and each reflection is a real Responses API call. The score

$$1000I_{\text{exact}} + 100q_{\text{facts}} - 5N_{\text{req}} - N_{\text{tool}} - N_{\text{tok}}/10,000$$

makes exact factuality dominant over the available efficiency terms. After optimization,

all five conditions run once per held-out issue under a balanced condition-order schedule. Because GEPA retains its seed, the nominal GCS+GEPA arm is an order-balanced replication of GCS, not evidence of a distinct combined optimizer.

4.8 Demonstration suite: workflow shapes

The suite deliberately escalates difficulty. Demos A–C are the shapes an optimizer is usually shown: a linear read prefix, a permissioned retrieval whose ACL scope and index version are hard guard keys, and a coordinator whose handoff is a region barrier. Demo D is a real stdio MCP server whose tool effects are undeclared, so the catalog must fail closed. Demo E is the adversarial case: its region contains two entry-state branches and one *observation*-dependent branch (a second page is read only when the first reports more results), so the executed call count varies at run time; and every episode ends in an irreversible commitment, so a region can only ever be a prefix and *partial* compaction is the best available outcome. Demo E is additionally run with a semantically equivalent loop-bearing artifact, which the model-boundary adapter must refuse rather than half-execute, and against entry states whose declared intake schema has moved, which the hard guard must reject. Demo F exercises the route-specialization pass instead of region compilation.

4.9 Metrics and statistical analysis

The primary efficiency outcome is provider requests. Secondary outcomes are input, output, and total tokens; provider and wall latency; estimated cost; and tool calls. In the fixed-prefix ablation, quality requires the correct category, evidence label, issue number, valid summary shape, and exact tool contract. In the natural-order studies, quality instead requires exact source-grounded fields and label decisions and is independent of tool order. The expanded task contract additionally requires one safe, grounded call to each necessary read tool while allowing any order and any integer comment-body limit. Paired differences use the same issues. We report 10,000-sample paired bootstrap confidence intervals [14] and two-sided Wilcoxon signed-rank tests [46]; binary quality uses exact McNemar tests [31]. Signed-rank p -values use the exact distribution where no ties occur and a tie-corrected normal approximation otherwise, and table 5 labels which applies per row. One row is deliberately *not* given a p -value: every pair changes provider requests by exactly -3 , so the difference is deterministic and a rank test on it reports an artifact of degeneracy rather than evidence. Secondary p -values are descriptive and unadjusted. Determinism is reported as decision, tool-trace, and byte-exact answer agreement. Peak memory and runtime are measured for the provider-free NESTFUL compiler stage. For portfolio admission, quality failures and non-positive group utilities are separate binary endpoints. Their one-sided exact bounds split 5% error across two actions and two endpoints; mean utility ranks only actions that pass both gates. The 12-record prospective cohort is an outcome evaluation, not additional calibration evidence. Its paired bootstrap intervals and signed-rank tests are descriptive and unadjusted. The exploratory GCS comparison uses the same paired procedures. Its primary contrast is GCS minus the measured provider-visible macro; it reports exposed tool interfaces and internal source reads separately so interface fusion cannot masquerade as eliminated work. The bounded comparator uses the same exact quality and paired resource procedures. Host monotonic wall time is primary. Provider-span latency is retained but excluded from a comparison if the summed spans exceed the surrounding host wall time by more than 250 ms. Optimization requests, tokens, latency, and cost

Table 1. External NESTFUL compiler results. “Wrong” means a synthesized program executed but disagreed with the recorded held-out outputs; abstentions are explicit.

Measurement	Result
Executable basic-function episodes	1415
Expected producer in candidate set (recall)	5531/5746 (96.3%)
of which uniquely resolved	4636 (80.7%)
of which ambiguous (truth among many)	895 (15.6%)
Slots with no candidate	215
Candidate-edge precision	5531/6564 (84.3%)
Complete groundable windows	1207/1415 (85.3%)
Recurring families with support ≥ 5	32
Synthesized families	12/32
Held-out replay: pass / abstain / wrong	24 / 12 / 0
Maximum family support / gate minimum	26 / 92
Certifiable families	0
Peak memory / elapsed time	23.3 MiB / 5.98 s

are never mixed with held-out deployment metrics.

5 Results

The evidence separates three questions. First, can a trace be reconstructed and synthesized? Second, is there enough independent evidence to admit the result? Third, if admitted, does it improve the end-to-end workflow against a strong manual alternative? NESTFUL and API-Bank answer the second question with refusal (sections 5.1 and 5.2). The GitHub studies answer the third: a conservative two-read artifact passes 30/30, an aggressive three-read artifact records one factual miss, and the manual macro remains the practical baseline (sections 5.3 and 5.5). GCS removes the provider-visible interface gap, but a fairly placed manual program ties its structural efficiency (sections 5.6 and 5.7). The remaining sections retain the failed suffix pilot, harder workflow shapes, and prospective portfolio pilot because each constrains a different claim rather than adding another headline.

5.1 RQ1 and RQ3: provenance succeeds more often than certification

Across 1,415 executable episodes, GAC places the expected producer in the candidate set for 5,531 of 5,746 dependency slots (96.3%). That figure is *candidate recall*, not dependency reconstruction, and it is close to definitional: candidates are generated by value matching and the gold producer emitted the matched value, so it joins the candidate set whenever any admissible path exists at all. Consistently, there is not one slot in which a non-empty candidate set omitted the true producer — the 215 misses are exactly the 215 slots with no candidate. Recall here therefore measures search *reachability* and is

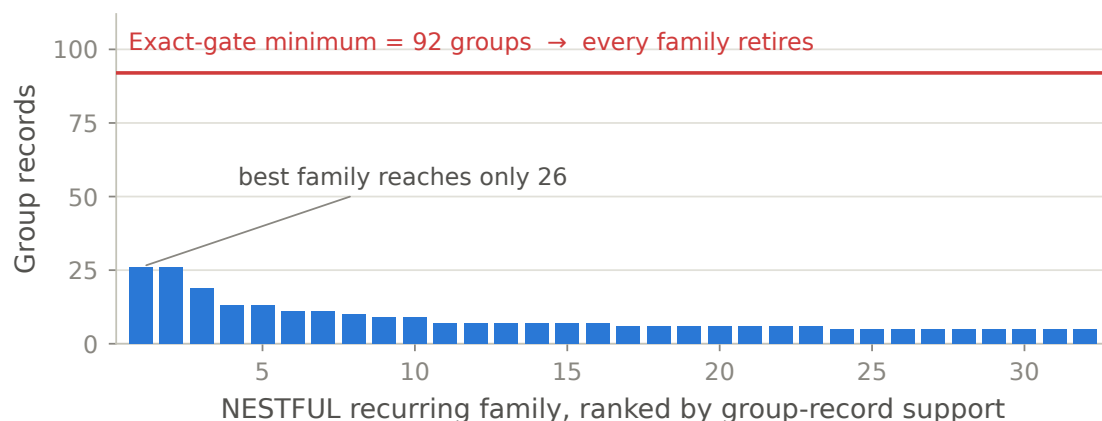


Figure 2. All NESTFUL families fall below the 92-group exact-gate requirement; the GitHub study was deliberately sized to reach it.

insensitive to ranking quality; it cannot fall below the groundability rate however poor the ranking. The load-bearing numbers are the other two: only 4,636 slots (80.7%) are resolved to a unique producer, 895 (15.6%) contain the expected producer among several candidates, and 215 have no candidate at all. Counting all emitted candidate edges gives 84.3% precision (5,531 of 6,564). H1 was pre-specified internally as recovery above 90%; it is supported on recall and *not* supported on unique resolution, and we report both rather than the more favourable one. It finds complete groundable windows in 1,207 episodes (85.3%). The remaining full-window failures are overwhelmingly ambiguous value matches (207), with one ungrounded slot. That count is per *episode* under first-hit attribution, and it is not the same denominator as the 215 no-candidate *slots* in table 1: an episode is attributed to the first reason that blocks it, so an episode containing both an ambiguous slot and an ungrounded one is counted as ambiguous, and slots outside every enumerated window are counted in neither. The two numbers are therefore consistent but not comparable, and we report both rather than the one that reads better. Recurrence is fragmented: 714 compiler families exist, only 32 have support at least five, and maximum support is 26.

Of 32 attempted recurrent families, 12 synthesize. Their 36 held-out windows yield 24 passes, 12 abstentions, and zero wrong executions. The dominant synthesis failures are unsupported loop predicates and slots that fail grouped refitting. This zero-wrong result does not certify the programs: the exact gate requires 92 zero-violation groups, so all families retire. H4 is supported, and the negative result distinguishes a guarded compiler from a recurrence-only macro miner.

Two properties of this benchmark path bound how far it generalizes. Its per-family split is lexicographic by episode identifier rather than randomized or chronological, so it tests held-out *windows* but not robustness to a shifted distribution; and the development partition it allocates is reported but not consumed, because synthesis reads the training windows and replay reads the test windows directly. This path also builds provenance, mining, synthesis, and replay itself rather than invoking the full compiler, so it is an executable structural benchmark, not end-to-end validation of the deployed gate.

The provider-free NESTFUL analysis completes in about six seconds, including roughly 1.4 for provenance construction, with 23.3 MiB peak traced memory. These numbers characterize this pinned dataset and Python process, not distributed produc-

Table 2. All ten named benchmark families receive an explicit executable disposition. Rows are deliberately not averaged: a gold-plan checker, a task adapter, a simulated live-provider run, and a post-trace compiler experiment license different claims.

Benchmark	Path exercised	Scope	Observed result / boundary
NESTFUL	Compiler	1,415 traces	24 / 12 / 0 held-out; all retire
API-Bank	Compiler + API replay	212 traces	0 / 2 / 0; RETIRE; upstream 338/389 exact
BFCL v4	Official gold checker	200 tasks	200/200 valid; no model score
ToolSandbox	Live-provider subset	1 scenario	milestone 0.982; compiler not run
τ^2/τ^3	Live-provider subset	4 tasks	0/4 pass; compiler not run
ToolBench	Adapter smoke	10 fixtures	full data/backend unavailable
AgentBench	Adapter + preflight	556 tasks	services and Freebase data gated
GAIA	Access preflight	0 downloaded	authorization denied; no metric
SWE-bench Verified	Task adapter	500 issues	official run host-gated
BrowseComp	Live-web subset	3 tasks	1/3 correct; 28 searches; bypass

tion throughput.

5.2 RQ6: broad coverage reveals narrow compiler eligibility

The all-source audit supports RQ6’s premise: popular agent benchmarks are useful, but most do not preserve the complete observed trajectory required for trace-derived replacement. Across eight accessible task adapters we normalize 5,419 tasks and 17,836 reference actions, yet only API-Bank’s 212 tasks contain complete observed call results. NESTFUL’s separate executable path supplies the other compiler substrate. The remaining rows still test parsers, upstream contracts, provider compatibility, or a conservative bypass, but their missing values are not counted as compiler failures.

API-Bank contains 389 observed calls over 49 tools. Conservative effect barriers leave 48 candidate windows in 37 tasks and 19 families; eight families have support at least three, but maximum support is only eight. Two families synthesize. Their two held-out windows produce zero passes, two abstentions, and zero wrong executions. The configured exact gate requires 92 independent zero-violation groups and therefore retires every family. This second corpus strengthens, rather than weakens, the central negative result: recurrence and trace completeness still do not create admission evidence.

Re-executing the pinned API implementations with fresh per-task fixtures exactly reproduces 338 of 389 recorded calls. Thirty-seven differ, nine raise missing-state Key-Errors, and five require unavailable dependencies; 162 of 212 tasks replay completely. These upstream data/code compatibility failures are reported separately from compiler replay. They also show why a recorded result and an executable current fixture are dif-

Table 3. Expanded natural-order Tier-2 replication on 30 fresh public issue records with live provider calls and all six condition orders balanced five times. Means use primary pairs only. Exact pass checks source-grounded fields; every arm also passes the safe-read task contract 30/30.

Condition	Requests	Tools	Tokens	Wall (s)	Cost	Exact pass
Unchanged agent	4.0	3.0	4259.4	6.16	\$0.000872	30/30
GAC (two-read prefix)	2.0	3.0	2576.5	2.97	\$0.000593	30/30
Hand-written macro	2.0	1.0	1780.8	3.25	\$0.000545	30/30

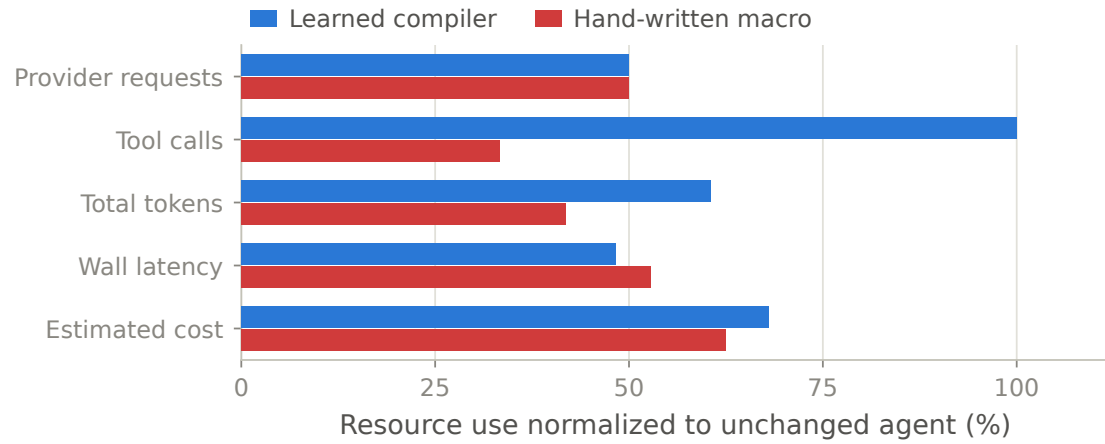


Figure 3. Expanded real-provider resource use relative to the unchanged agent. The partial compiler pre-executes two reads but retains all three tool calls; the macro combines the reads into one.

ferent evidence objects.

The bounded provider paths reveal cost and quality failure modes without changing that conclusion. The four τ runs account for 71 provider responses with usage and the three BrowseComp tasks account for six requests; ToolSandbox’s pinned harness omits usage, so its three assistant messages are not relabeled as exact request accounting. In total, 77 provider requests are exactly accounted and ToolSandbox remains explicitly incomplete. No prompt, answer, tool argument, tool result, search result, or credential from these runs is serialized into the publication artifacts.

5.3 RQ2: real-provider efficiency and observed quality

5.3.1 Natural tool ordering, compilation depth, and a live macro.

In the expanded replication, all 132 discovery traces read the record, then labels, then comments without the prompt naming tools or order; 130 pass the exact-source task. The compiler does *not* blindly turn recurrence into a macro. It rejects the full three-read candidate because the comments limit has no consistent trace-grounded expression, then emits the groundable two-read prefix. The gate admits 92/92 calibration groups with zero registered violations at $\alpha = .05$, $\delta = .10$, and 11 thresholds, for simultaneous upper bound 0.0498. Thresholds below 0.11 admit none and those at or above 0.11 admit all, so this remains a sample-size gate rather than a demonstrated risk-coverage discriminator.

Relative to the unchanged agent, the partial compiler reduces requests 50.0%, total tokens 39.5%, observed wall latency 51.7%, and estimated cost 32.0%, while leaving

Table 4. Earlier aggressive natural-order study on 18 fresh public issues. The complete three-read artifact saves more provider turns but records one exact-source miss.

Condition	Requests	Tools	Tokens	Wall (s)	Cost	Factual pass
Unchanged agent	4.0	3.0	4065.4	5.69	\$0.000787	18/18
GAC	1.0	3.0	1381.0	1.34	\$0.000404	17/18
Hand-written macro	2.0	1.0	1604.9	4.68	\$0.000459	18/18

three necessary reads. The macro also halves requests, but reduces tokens 58.2%, cost 37.5%, and tool calls 66.7%. It therefore beats the learned artifact on tools, tokens, and dollars; the compiler has lower observed mean wall time (2.97 versus 3.25 seconds), but its paired 95% interval against the macro crosses zero. Every primary arm passes 30/30 exact factual and full task contracts. With zero compiler-only failures, the one-sided 95% exact upper bound on the sample’s underlying discordance rate is still 9.5%; this is observed preservation on one domain, not equivalence or non-inferiority.

Ten records are repeated once per arm. Category decisions agree in 10/10 for all arms, but byte-exact answers agree in 7/10 baseline, 6/10 compiled, and 4/10 macro pairs; tool traces agree in 9/10, 9/10, and 10/10. Compaction therefore does not improve free-text determinism here. The provider-free semantic regrade changes no output, token, timing, or cost measurement: it removes a planted literal comment-limit requirement that contradicted the “as needed” prompt, preserves all 212 online quality objects, and binds the final result to the paid discovery checkpoint by SHA-256.

The earlier 18-record follow-up tested the more aggressive three-read artifact and remains important negative evidence.

Relative to its unchanged agent, that artifact reduces requests 75.0%, total tokens 66.0%, observed wall latency 76.4%, and estimated cost 48.7%; its macro reduces the same quantities by 50.0%, 60.5%, 17.7%, and 41.7%. The unchanged agent and macro pass 18/18 factual contracts, but GAC passes 17/18. On issue 6602 the source contains a Markdown link while the compiled continuation drops the URL and returns only its anchor text. The single discordance gives two-sided exact McNemar $p = 1$ and a one-sided 95% exact upper bound of 23.8%. H3 is therefore supported as an observed-sample result for the expanded two-read artifact and not supported for the earlier three-read artifact; preservation is not invariant to compilation depth.

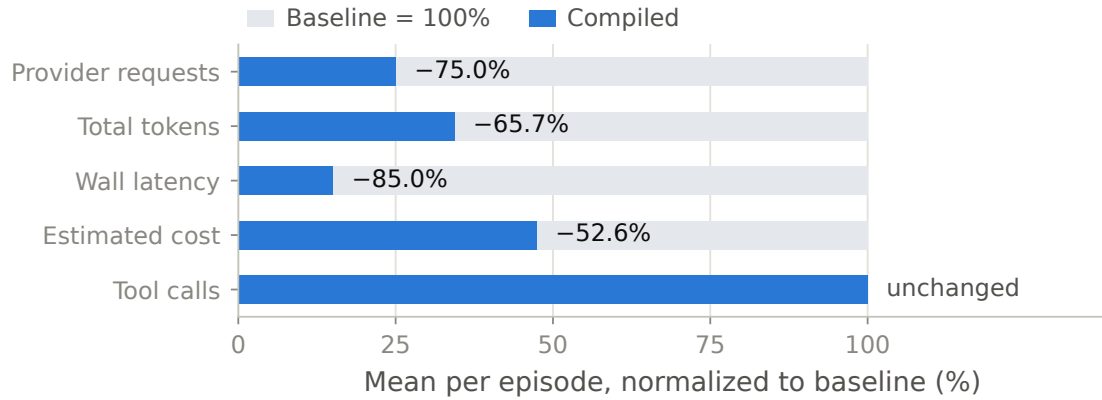
The miss fixes the gate’s scope. Zero calibration violations concern the compiled *tool program*: grounded arguments, tool outputs, call counts, and declared effects. The verifier does not certify the downstream model continuation. Thus 45/45 clean program replays and a held-out factual-output miss are compatible. A deployment claim requires a second admission layer over continuation outcomes or a deterministic checked renderer.

We implemented that boundary as a framework-neutral ContinuationGuard. It checks the candidate answer, then tries an optional deterministic renderer and baseline in that order, revalidating either output before release; invalid recovery returns no output. A provider-free replay accepts 17 retained compiled answers, detects issue 6602, and checked-renders it from source observations, yielding 18/18. This post-hoc mechanism check establishes detection and rendering on retained records, not live latency, cost, non-inferiority, or cross-domain safety.

That earlier study’s online grader also imposed an undocumented excerpt mini-

Table 5. Paired results on 18 unseen real issues. Change is compiled relative to baseline. p_W is the unadjusted Wilcoxon signed-rank p -value.

Metric	Baseline	Compiled	Change	p_W
Provider requests	4.0	1.0	−75.0%	deterministic
Tool calls	3.0	3.0	−0.0%	1
Input tokens	3798.5	1294.7	−65.9%	2.0e-04
Output tokens	119.3	50.6	−57.6%	1.9e-04
Total tokens	3917.8	1345.2	−65.7%	2.0e-04
Wall latency	10.29 s	1.55 s	−85.0%	7.6e-06
Estimated cost	\$0.000784	\$0.000372	−52.6%	7.6e-06
Task-quality pass rate	1.000	1.000	0.0 pp	1.0

**Figure 4.** Mean compiled resource use normalized to baseline. Necessary local reads are unchanged; provider-mediated decisions decrease.

mum and misread a literal comment “none” as a sentinel. Its separate provider-free revision corrected four rows, retained online labels and the prior digest, and changed no provider metric. It did not retrain the executed artifact; after correction issue 6602 remains the only compiled-only failure.

5.3.2 Prescribed-prefix ablation.

The compiled GitHub artifact has 16/16 training-group support, passes all eight development replays, and admits 92/92 calibration groups with zero observed violations. At $\alpha = .05$, $\delta = .10$, and 11 thresholds, the simultaneous upper bound is 0.0498. The artifact was promoted only under a paper-protocol lab flag; no production canary is claimed.

Provider requests fall from exactly 4 to 1 per issue (75.0%; paired 95% bootstrap difference $[-3, -3]$), supporting H2. Mean total tokens fall from 3,917.8 to 1,345.2 (65.7%); wall latency from 10.29 to 1.55 seconds (85.0%); and estimated cost from \$0.000784 to \$0.000372 (52.6%). All three paired bootstrap intervals exclude zero; appendix D reports their endpoints. Every held-out issue is cheaper, faster, and lower-token under compaction (fig. 5).

Tool calls remain exactly three in both arms. GAC removes redundant model choices,

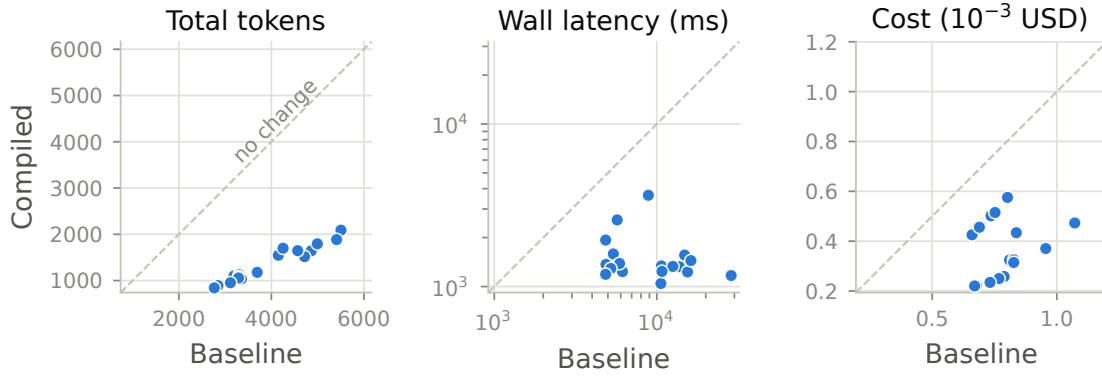


Figure 5. Per-issue paired measurements. All points fall below the no-change diagonal for total tokens, wall latency, and estimated cost.

not evidence acquisition. All 18 baseline and compiled runs pass the composite quality contract. This supports H3 only as an observed sealed-sample result. With no discordant pairs McNemar’s exact test returns $p = 1$, which carries essentially no information about equivalence and should not be read as evidence of it. The informative quantity is the one-sided bound on the paired degradation rate: treating “baseline succeeds, compiled fails” as a Bernoulli event with zero occurrences in 18 pairs, the exact 95% upper bound is $1 - .05^{1/18} = 15.3\%$. Perfect observed agreement here is therefore compatible with an operationally unacceptable degradation rate, and no interval on tokens or latency repairs that. A predeclared non-inferiority margin of a few percentage points would need hundreds of sealed pairs per domain.

Discovery itself is not free. The 132 discovery episodes consumed 528 provider requests, 533,293 tokens, 1,109 seconds of summed wall time, and an estimated \$0.0958. The confirmatory baseline and compiled arms cost an additional \$0.0170 and \$0.0076 over 24 runs each (including repeats). Break-even depends on which resource is amortized, and reporting a single number selects the most favourable one, so we report the range rather than a point: 176 eligible future episodes when amortizing provider requests, 207 on tokens, 232 on discovery dollars alone, and 292 once the confirmatory arms that were actually incurred are counted.¹ All four are lower bounds twice over. They exclude engineering, review, and monitoring effort, which for a one-artifact deployment plausibly dominates every figure here; and the dollar rows are computed at list input price, whereas section 5.4 measures baseline cached-input shares of 96–98% on comparable workloads. If the four-turn baseline here is similarly cache-dominated — and a repeated loop over one long prefix almost certainly is — then both the −52.6% cost reduction in table 5 and every dollar break-even above are optimistic, and the correction is the two-tier price model of section 7.3. We report the numbers we measured and the direction of their bias rather than re-deriving them from an unmeasured cache model.

5.4 Harder shapes, partial compaction, and where the saving inverts

Table 6 reports the suite, ordered by role rather than by letter: the five compacting conditions first, then the three whose correct outcome is refusal. Three cells need explanation before the findings. Demo D has $n = 2$ because the MCP negative control defines only two tenant scenarios, and adding more would repeat one refusal rather than test an-

¹528/3; 533,293/2,572.6; \$0.0958/\$0.000412; \$0.1204/\$0.000412.

Table 6. Demonstration suite (Tier 3): live provider calls and the real SDK runtime against deterministic local services. Negative change is a saving. The bottom three rows are negative controls whose correct outcome is *no* compaction; each returns exactly the baseline model-call count at unchanged quality, supporting H5. Their dollar columns differ from baseline despite identical work: see the cache discussion below.

Demo	Shape that makes it hard	n	Model calls		Change		Quality
			base	cand.	tokens	cost	base / cand.
A	Linear read prefix	4	6.0	1.0	−79.6%	−73.6%	1.00 / 1.00
B	ACL scope + index version as guard keys	4	7.0	1.0	−76.3%	−68.7%	0.97 / 0.97
C	Coordinator with a handoff barrier	4	5.0	1.0	−73.8%	−75.5%	1.00 / 1.00
E	3 synthesized branches, pagination, mandatory write	4	7.0	2.0	−66.4%	+8.3%	1.00 / 1.00
F	Route specialization (prompt + tool surface)	4	7.0	7.0	−16.8%	+123.0%	1.00 / 1.00
D	Undeclared MCP effects (negative control)	2	3.0	3.0	+0.1%	+0.2%	1.00 / 1.00
E′	Loop-bearing artifact refused (negative control)	4	7.0	7.0	+0.2%	+55.4%	1.00 / 1.00
E″	Entry-schema drift, guard miss (negative control)	4	7.0	7.0	+0.0%	+54.5%	1.00 / 1.00

other. Demo B scores 0.97 in *both* arms: one of its four scenarios returns four citations where the fixture expects five, in baseline and compacted alike, so it is a property of the workload rather than of compaction — and it is the one place in the suite where the graded score is not 1.00. Demos E′ and E″ share Demo E’s world and differ only in the artifact offered to the runtime. With that said, three results matter beyond the headline.

Partial compaction is the correct outcome when a workflow commits. Demo E ends every episode in an irreversible write, so its region is bounded by eq. (2) and the commitment itself. Model calls fall from 7.0 to 2.0 rather than to 1.0: the evidence turns disappear, and the decision turn and the write survive. Tokens fall 66.4% at unchanged task quality. This is the shape most real operational workflows have, and it is absent from Tier 1 and Tier 2; reporting only demos that end in a read would overstate what the method reaches.

Refusal reproduces the baseline execution, not the baseline invoice. The three negative controls — undeclared MCP effects, a loop-bearing artifact the adapter cannot advance one turn at a time, and an entry state whose intake schema has drifted past the guard’s pin — all return 7.0, 7.0 and 3.0 model calls against baselines of 7.0, 7.0 and 3.0, with quality identical to baseline in every case. The runtime therefore pays the guard and gate, not the workflow, when it declines. A compiler that silently degraded instead would be indistinguishable from these rows on the efficiency metrics alone, which is why H5 is stated separately.

Their *dollar* cost is not identical, and table 6 shows why the distinction must be explicit: two refusing conditions bill 55.4% and 54.5% more than their baseline on identical token counts. Nothing about the refusal changed the work; the runs differ only in prompt-cache warmth, because each condition executes as a separate batch and the

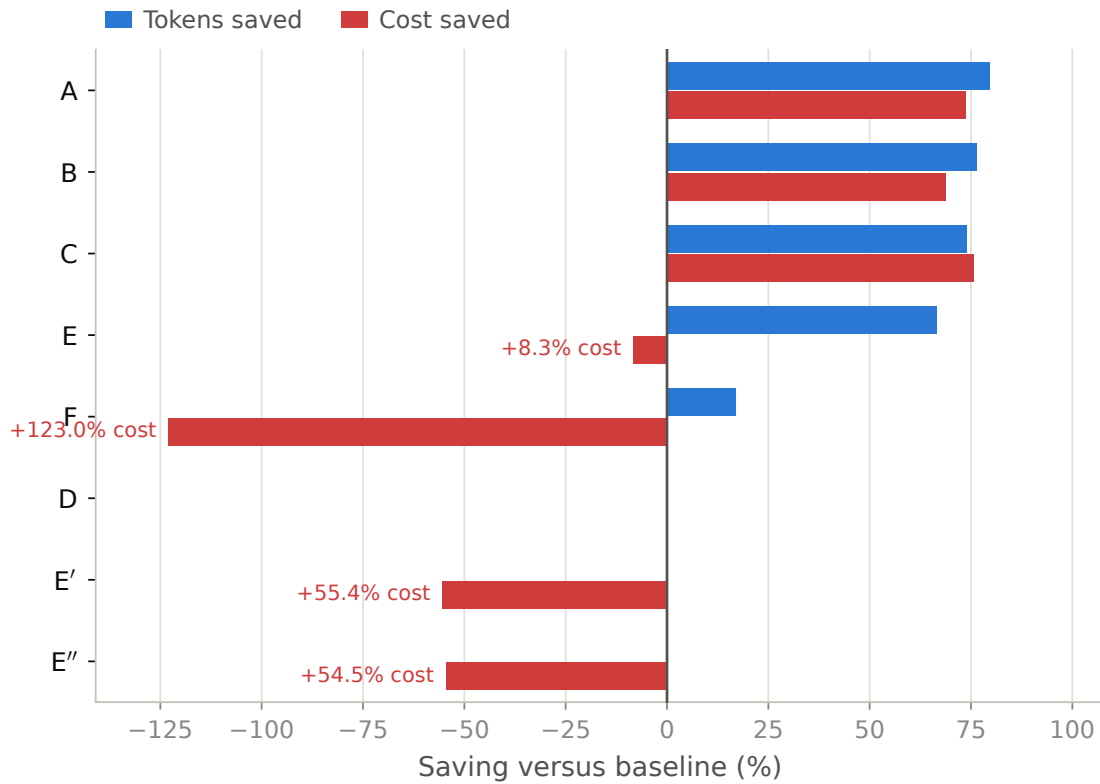


Figure 6. Token saving against cost saving per demonstration. The two diverge, and for Demos E and F they take opposite signs: prompt-cache economics, not token count, decide the invoice.

baseline batch runs first (see the ordering caveat in section 8). Reporting H5 in dollars would therefore record a provider-side scheduling artifact as a property of the compiler.

Token saving and cost saving can have opposite signs. Demo F reduces tokens by 16.8% and *increases* estimated cost by 123.0% (fig. 6). Demo E shows the same sign flip at +8.3%, but we do not rest anything on it: the refusing conditions E' and E'' do *identical* work and still bill +55.4% and +54.5%, so this run's ordering artifact is worth several times Demo E's effect and swamps it. Only Demo F clears that floor, and even there the magnitude should be read as indicative. The mechanism we claim is prompt caching: a seven-turn baseline pays one cache *write* over a long prefix and then reads that prefix at a fraction of the input price six more times, whereas a two-turn compacted run pays the write with almost nothing left to amortize it against. Route specialization makes it worse one level up, because each route is a distinct cache prefix and a fleet that shared one warm prefix now pays several writes. Measured cached-input share falls from 96% to 38% (Demo E) and from 98% to 58% (Demo F).

The direction is what the data supports; the magnitude is not, and the honest floor is set by the refusing conditions rather than by our own confidence. The effect also shrinks as episodes per prefix grow, so a suite this small understates the steady-state cost advantage of both techniques. But the residue is a real design constraint that our objective in eq. (5) does not price: a rarely-exercised route may never amortize its own cache write, and a workload whose baseline is already cache-dominated has far less dollar headroom than its token headroom implies. Reporting token reduction as though it were cost reduction — as an optimizer that measured only tokens would — is therefore unsafe, and section 7.3 lists the fix.

Table 7. GAC against the obvious engineering alternative — one hand-written composite tool performing the same reads — on the deterministic offline stress study. R_{req} is the paired request ratio against the unchanged agent, so lower is better and 1.000 means no saving. **The macro is at least as good on three of five workloads.** This substrate is simulated: a scripted policy stands in for the model at each boundary, so these numbers cannot settle the live question. They can, and do, settle whether the comparison was run.

Demo	Workload	Macro R_{req}	GAC R_{req}	Lower is better
A	Tier-1 support evidence gathering	0.726	0.755	macro wins
B	permissioned RAG knowledge assistant	0.335	0.718	macro wins
C	multi-agent incident triage	0.922	0.780	GAC wins
D	multi-tenant MCP operations (negative control)	0.724	1.000	macro wins
E	order-fulfillment exception handling	0.661	0.642	GAC wins

5.5 The comparator that matters: a hand-written composite tool

For a workflow whose reads are already known, the obvious engineering answer is not a compiler: it is to write one composite tool that performs them. A request reduction measured only against an unoptimized agent cannot distinguish “compilation works” from “this workflow was easy”. Table 7 therefore reports both against the same baseline on the offline stress study, and the result is not in our favour.

On the permissioned-retrieval workload the macro is far better ($R_{\text{req}} = 0.335$ versus 0.718): it collapses six reads into one call, whereas GAC must keep each call separately dispatchable to verify its live-outs. On Tier-1 support the macro is slightly ahead. GAC wins where the *choice* of which reads to make is itself part of the recurrent structure — multi-agent triage (0.780 versus 0.922), where a composite tool cannot remove the coordinator’s routing turn, and fulfillment (0.642 versus 0.661), where branch-dependent evidence means a single fixed composite would over-read.

The negative control is the most informative row. A hand-written macro reaches $R_{\text{req}} = 0.724$ on the MCP workload; GAC reaches 1.000, because the tool effects are undeclared and the catalog fails closed. The macro takes a 27.6% saving that GAC refuses. Whether that refusal is prudence or lost value depends entirely on whether those undeclared tools really are read-only — which is exactly the question a human answers by writing the macro and the compiler declines to guess. Neither number is “right”; the pair is the point.

We therefore do not claim that GAC dominates ordinary engineering. The expanded live comparison in table 3 reaches the same conclusion directly: both candidates pass 30/30, but the macro matches the partial compiler’s request count and uses fewer tools, tokens, and dollars; only observed wall latency favors the compiler. The earlier aggressive artifact in table 4 saves one additional provider turn but records a factual miss. Where a hand-written composite is available and its effects are understood, write it. GAC earns its complexity only when the region is not economically maintainable as a manual macro, is branch-dependent, or requires guards a macro would otherwise assume implicitly.

The result has a direct mechanistic explanation and a testable hypothesis. The macro is allowed to redesign the application interface: it fuses three reads, returns a task-specific sufficient record, and exposes one tool schema. The compiler is constrained to preserve existing interfaces, provenance, intermediate evidence, and fallback seman-

Table 8. Fresh real-record comparison after adding guarded composite synthesis. “Exposed” counts model-visible interfaces; both conditions execute three source reads. GCS runs the admitted composite before the first provider request. This is a 12-pair exploratory extension designed after the earlier macro result, not a confirmatory replacement for table 3.

Condition	Requests	Exposed	Reads	Tokens	Wall (s)	Cost	Exact
Provider-visible macro	2.0	1.0	3.0	1524.0	2.49	\$0.000430	12/12
Pre-model GCS	1.0	1.0	3.0	931.2	1.49	\$0.000291	12/12

tics; in the expanded run its groundability proof permits only a two-read prefix and all three logical reads remain necessary. We therefore hypothesize that manual macros dominate on stable, low-entropy workflows, while guarded compilation earns its cost across branching or changing workflow families for which manual discovery and maintenance do not amortize. This study establishes the structural facts behind that hypothesis, but it does not measure engineering effort, drift response, or the cross-workflow break-even.

5.6 Exploratory GCS: closing the measured macro gap

We implemented the interface transformation suggested by the comparator rather than discarding the guard. The compiler now canonicalizes only application-declared task-semantic arguments, emits the complete three-read program behind one projected interface, retains all internal provenance and verification, and executes it before the provider only when the continuation manifest matches exactly. Provider-free reconstruction of all 132 retained discovery traces dispatches 124, safely falls back on eight, and produces 124/124 exact projections with no projection failure. That replay makes no new model call and supports implementation correctness, not an efficiency claim.

The paid comparison uses 12 further public issues excluded from all 424 issue identifiers used by the preceding experiments. Both the provider-visible hand-written macro and GCS pass all 12 exact factual and tool contracts; with no observed discordance, exact McNemar $p = 1$, and the one-sided 95% zero-event bound still permits a 22.1% population discordance rate. Relative to the measured macro, GCS reduces provider requests from 2.0 to 1.0 (−50.0%), total tokens from 1,524.0 to 931.2 (−38.9%), mean wall latency from 2.49 to 1.49 seconds (−40.0%), and estimated cost from \$0.000430 to \$0.000291 (−32.3%). Paired request, total-token, latency, and cost differences all have two-sided exact signed-rank $p = 0.000488$; output-token reduction alone is uncertain ($p = 0.155$). Each condition exposes one interface and still performs three source reads, so this is decision and context compaction, not elimination of necessary evidence.

The mechanism is placement, not magic: the provider-visible macro spends one request choosing its composite and another producing the answer, whereas GCS supplies the guarded projection to the first request. The follow-up in section 5.7 implements the previously missing manually engineered, equally guarded *pre-model* comparator. The run also covers only five bug and seven other-labelled issues—no question or enhancement cases survived strict prior-cohort exclusion. A retained two-case smoke run had worse GCS latency, reinforcing that the 12-case latency magnitude is provider- and schedule-sensitive. The supported conclusion is therefore precise: automatic guarded interface synthesis matches the measured macro’s observed quality and beats its measured provider work on this family, while preserving safe fallback. It is not evidence of an advantage over manual placement.

Table 9. Fresh real-record deployment after bounded prompt optimization. Values are means over six held-out issues. “Interfaces” counts model-visible tool calls. GEPA retained its seed; the combined row is therefore a GCS replication, not a distinct optimized prompt. Optimization overhead is excluded.

Condition	Requests	Interfaces	Input	Total	Wall (s)	Exact
Unchanged	4.0	3.0	3542.0	3683.5	5.08	6/6
GEPA (seed retained)	4.0	3.0	3530.0	3669.3	4.33	6/6
GCS	1.0	1.0	770.8	861.0	1.61	6/6
GCS + retained seed	1.0	1.0	770.8	868.8	1.78	6/6
Manual pre-model	1.0	1.0	770.8	865.5	1.46	6/6

5.7 Fair placement and bounded prompt optimization

All five conditions pass all six exact factual and tool contracts. Relative to the unchanged agent, GCS reduces provider requests 75.0%, exposed interfaces 66.7%, input tokens 78.2%, total tokens 76.6%, observed wall time 68.2%, and estimated deployment cost 67.9%; the paired structural and token reductions have exact signed-rank $p = .03125$. These six pairs remain too few for a population equivalence claim.

The fair manual program is the decisive comparator. It also uses one request, one exposed interface, and 770.8 mean input tokens, and passes 6/6. Against it, GCS changes none of those structural metrics. GCS emits 4.5 fewer output and total tokens on average ($p = .25$), while observed wall time and cost differences are also non-significant ($p = .844$ and $.625$). Automatic discovery, provenance, compatibility pins, and calibrated admission may justify GCS operationally, but this study shows no runtime dominance over a correctly placed hand-authored program. Manual construction and review effort were not measured.

GEPA consumes 14 real task evaluations and three real reflection calls under the fixed budget, proposes three alternatives, and retains the seed. Its deployment arm therefore keeps four requests and three interfaces; total-token reduction versus unchanged is 0.38% ($p = .688$), and estimated cost is 3.6% higher ($p = .688$). Optimization itself consumes 59 provider requests, 63,954 tokens, and an estimated \$0.01163, reported outside deployment. Because no prompt change is selected, the combined arm cannot support a composition claim. This is a bounded negative result on one extractive workflow, not a general failure of GEPA. One GCS+seed provider-span record exceeds its surrounding host wall time; the raw span is retained, but provider-span comparisons involving that arm are invalidated.

5.8 RQ5: prospective portfolio selection

Both actions record zero quality failures and zero non-positive-utility groups among 30 independent calibration issues. With the multiplicity split, each one-sided upper bound is 0.1359, below the pilot limit 0.15. The compiler’s mean utility is 0.327 and the macro’s is 0.489, so the selector emits a macro recommendation and requires explicit review. The decision, objective weights, compatibility key, source-result digest, and fresh issue identifiers are stored before provider execution.

On the 12 calibration-disjoint issues, baseline and selected macro both pass 12/12 exact factual and full task contracts with no infrastructure failure. The selected action changes every workflow from four to two provider responses and from three to one

Table 10. Risk-bounded portfolio calibration and fresh-cohort outcome. $\bar{U}$ is the weighted mean of paired reductions in cost, latency, tokens, and tool calls. Both calibration bounds split 95% familywise confidence across two actions and two endpoints. The fresh cohort is evaluation evidence, not additional calibration.

Calibration action	n	$\bar{U}$	R_q^+/R_u^+	Decision
Compile	30	0.327	0.136/0.136	admitted
Macro	30	0.489	0.136/0.136	selected; human review

<i>Fresh cohort: selected macro versus unchanged</i>				
Endpoint	n	Base	Macro	Change
Exact contracts	12	12/12	12/12	preserved
Provider requests	12	4	2	−50.0%
Tool calls	12	3	1	−66.7%
Total tokens	12	–	–	−59.2%
Estimated cost	12	–	–	−40.6%
Wall latency	12	–	–	−71.6%

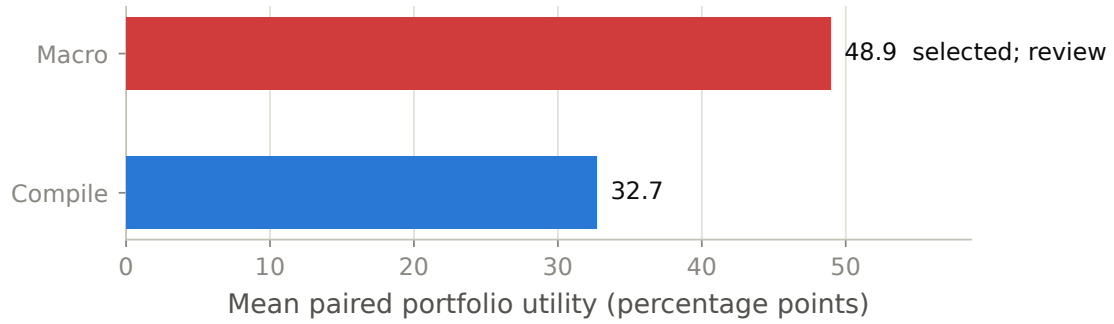


Figure 7. The frozen portfolio evidence admits both measured actions but recommends the higher-utility macro for human review before the fresh records are selected.

tool call. Aggregate reductions are 59.2% for total tokens, 71.6% for observed wall latency, and 40.6% for estimated cost. The paired macro-minus-baseline intervals are $[-2831.0, -2203.3]$ tokens, $[-12.174, -3.706]$ seconds, and $[-\$0.000445, -\$0.000267]$ per issue; all corresponding unadjusted signed-rank p -values are 0.00049. H_6 is supported on its observed-sample terms.

This result is more informative than another retrospective arm because the action is fixed before the test records are chosen. It is not a general portfolio demonstration: one workflow family supplies both calibration and test, no cache candidate has measured evidence, and the selected action is the same macro that a static always-macro policy would choose. Showing value from *selection* requires multiple families on which different actions are optimal, measured construction and maintenance costs, and time-forward drift.

5.9 RQ4: failed pilot and determinism

The first live pilot compiled suffix-only regions even though the runtime resolves only at entry. It therefore dispatched label/comment reads before their intended position, duplicated or reordered tools, and achieved only 16.7% task/tool-contract validity despite reducing requests. We archived the complete pilot, added the prefix invariant in eq. (2), added a regression test, and excluded every pilot issue from the final cohort.

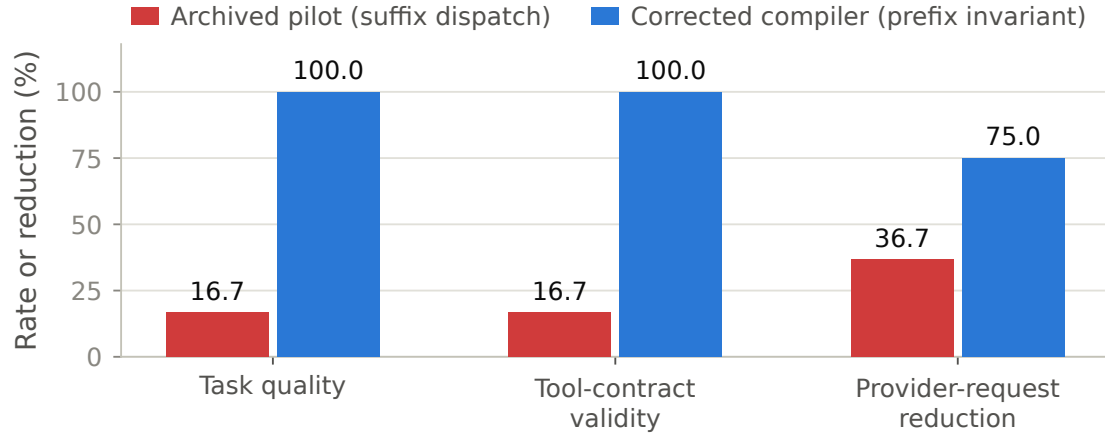


Figure 8. Safety ablation from the archived pilot. Request savings without correct execution position are not an acceptable optimization.

The corrected compiler blocks 116 suffix candidates and restores 100% observed contract validity (fig. 8).

On six repeated test issues, category decisions and tool traces agree perfectly in both conditions. Byte-exact natural-language answers agree in one of six baseline pairs and zero of six compiled pairs. Compaction therefore improves structural determinism of the tool prefix by construction, but not free-form surface determinism. This small sample does not support a claim that it stabilizes final text.

6 Related Work

Agent execution and scheduling. ReAct interleaves reasoning with environment actions [48]. LLMCompiler generates a dependency plan for the current request and schedules independent calls in parallel, reporting up to $3.7\times$ latency and $6.7\times$ cost improvement over ReAct [24]. GAC instead learns from repeated executions and deletes historical model boundaries; it currently executes synthesized calls sequentially. The methods are complementary.

Workflow and agent optimization. DSPy and MIPRO optimize instructions and demonstrations in declarative LM programs [23, 36]. RouteLLM selects a model based on preference data [33]; AgentSlimming prunes or replaces multi-agent graph nodes and reports token reductions up to 78.9% [9]. FlowCompile explores model, reasoning-budget, and workflow configurations for a declared graph and reports up to $6.4\times$ speedup [25]. These optimize parameters or declared topology, whereas GAC infers a value-grounded replacement from observed traces.

GEPA uses execution and evaluation traces as natural-language feedback for reflective, instance-wise Pareto prompt evolution [2]. Across its six reported tasks it improves over GRPO with up to $35\times$ fewer rollouts and produces instructions up to $9.2\times$ shorter than MIPROv2 prompts. GEPA therefore prevents us from claiming trace-driven agent optimization itself as novel. It changes residual prompts rather than deleting model boundaries, making it complementary to region compilation. Our bounded factorial comparison covers unchanged, GEPA-only, GCS, combined, and manual pre-model conditions on six held-out records. Official GEPA 0.1.4 retains its seed, so the combined arm is a GCS replication rather than evidence of interaction. This budget-limited negative result does not contradict GEPA’s broader task results.

AWO is the closest trace-based system: it identifies recurring tool-call sequences and bundles them into meta-tools, reducing LLM calls by up to 11.9% [1]. Agent Workflow Memory induces reusable workflows offline or online and reports large relative success gains on Mind2Web and WebArena [43]; it optimizes *what* an agent reuses rather than proving a reuse admissible, and it is the learned-workflow comparator a future head-to-head most needs. Agentic Compilation generates a deterministic JSON workflow from a single model call and replays it without further inference, reporting 80–94% zero-shot compilation success on web automation [12]; it shares our compile-versus-rerun motivation and differs in admitting the compiled plan without provenance, effect, or finite-sample risk conditions. A recent survey separates reusable templates, run-specific realized graphs, and traces [49]; in that taxonomy GAC optimizes traces under a declared-effect contract. EvoC2F compiles a typed Plan IR carrying dependency, effect, resource, idempotency, and retry annotations, then admits trajectory-derived macro-skills through tests, contracts, and regression checks [45]. Agent JIT Compilation generates and validates code plans for current web tasks, selects low-cost candidates, and schedules execution under tool pre/post-state invariants [47]. Both are closer compiler comparators than prompt-only optimizers. GAC differs by mining recurrent cross-execution regions from value-level traces and attaching a conditional group-level contract bound; the present experiments do not establish superiority to either system. Agentic plan caching adapts reusable plan templates and reports 46.62% mean cost reduction [50]. GAC adds explicit typed provenance, effect and position barriers, hard compatibility contracts, verifier/staging semantics, and an exact selective admission rule. Our result should not be read as a head-to-head improvement: we did not reimplement AWO on the GitHub scenario.

COVENANT compiles natural-language policies into workflow graphs and checks proposed actions against the declared procedure [42]. It uses policy as source; GAC uses executions as candidate evidence and must therefore abstain more aggressively. Combining declared workflow authority with trace-derived specialization is promising.

Tool-use evaluation. BFCL covers serial, parallel, abstaining, and stateful function calling [37]; API-Bank provides runnable APIs and tool-use dialogues [26]; ToolSandbox evaluates stateful conversational interactions [28]; τ^2 -Bench adds dual control by agent and user [5]; ToolBench scales tool retrieval [38]; AgentBench spans interactive environments [27]; GAIA and BrowseComp test general assistant and persistent browsing capabilities [32, 44]; and SWE-bench grounds agents in real software issues [21]. These primarily test whether an agent can choose and execute actions or produce a final answer. NESTFUL exposes nested producer references, while API-Bank retains recorded call results, making those two suitable for our post-trace question [6]. Our all-source audit executes or gates every named path (table 2) rather than using absent trajectories as negative examples. Stateful and write-bearing compilation remains future work until transactional staging exists.

Trace-based specialization, guards, and deoptimization. The architecture is, structurally, a tracing JIT for agent executions, and the correspondence is close enough that the vocabulary is nearly shared: hot-trace detection becomes family mining, guard synthesis becomes the hard guard, region compilation becomes bounded synthesis, and deoptimization to the interpreter becomes fallback to the baseline agent. Dynamo introduced transparent trace regions with guards and fallback [4], and trace-based type

Table 11. Positioning against adjacent optimization systems. “Admission” describes how a candidate rewrite is permitted, not its general evaluation protocol.

System	Source	Rewrite target	Hard safety	Admission
LLMCompiler	Current-query scheduling	Parallel tool calls	No	No
DSPy / MIPRO	Prompt/ program parameters	Metric optimization	No	No
GEPA	Execution + evaluation traces	Reflective prompt evolution	No	Held-out metric + Pareto
AgentSlimming	Multi-agent graph	Node pruning/ quantization	No	Baseline rule
AWO	Trace tool sequences	Deterministic meta-tools	No	Empirical
Agent JIT	Task description	Validated code + schedule	Tool pre/post invariants	Candidate validation
EvoC2F	Plan IR + trajectories	DAG compiler + macro-skills	Effects + resources	Tests + contracts
FlowCompile	Declared workflow	Configuration Pareto set	No	Profiled
COVENANT	Declared policy	Workflow CFG	Effects	Runtime checks
GAC (ours)	Observed value flow	Decision-eliding program	Provenance + effects	Exact selective bound

specialization [16] and meta-tracing [8] developed the guard-and-recompile discipline we borrow. The problem of section 3.7.1 — restoring a consistent state after a specialized region is abandoned — is exactly dynamic deoptimization [18], and that literature’s answer, explicit deoptimization points, is the same mechanism we defer to as “control points with continuation tokens”. Effect systems supply the discipline behind treating effect declarations as a trusted computing base [29], and selective prediction long pre-dates learn-then-test: the reject option is Chow’s [11]. What is not inherited is what an agent guard must additionally certify: that arguments are grounded in observable state, that declared effects permit speculation, and that the residual error rate carries a finite-sample bound. We take the framing as the clearer statement of the contribution.

Behavioral equivalence and caching economics. Counterfactual trace auditing pairs with-skill and without-skill traces, aligns their phases, and shows that unchanged aggregate pass rates can hide many substantive behavioral differences [51]. That is exactly the weakness of our registered-contract oracle, and it describes the instrument a stronger equivalence check would use. On economics, prompt-cache strategy has been evaluated across three providers on long-horizon agentic tasks, reporting 41–80% API-cost reductions and showing that naive full-context caching can even increase latency [30]; cache-aware prompt compression models the compression/caching crossover directly [40]. Our section 5.4 observation that fewer tokens can cost more is therefore *consistent with* that literature rather than a new discovery. What our data adds is the same effect arising from *turn removal and route fragmentation* rather than from compression, and the consequence that a workflow optimizer’s objective must

price prefix reuse.

Program analysis and selective risk. Canonical trace families resemble process-mining variants [41]; bounded synthesis relates to partial evaluation [22], likely-invariant mining [15], and programming by example [17]. Risk-controlled admission builds on exact binomial inference [13] and learn-then-test calibration [3]. The guarantee remains conditional on i.i.d. or conditionally i.i.d. admitted group indicators and a frozen candidate; provenance and effects are separate hard constraints, not learned away by calibration.

7 Discussion

7.1 What is novel, and what is not

Core novelty. The contribution is not agent compilation; it is *admissibility for trace-derived specialization*. Deterministic workflows [12], reusable trace patterns [1, 43], and effect-aware plans all have clear precedent. A classical JIT guard checks types and shapes. An agent guard must additionally show that arguments are grounded in observable state, declared effects permit speculation, runtime position is safe, and observed contract risk is bounded. GAC combines those requirements in one compile-or-retire path: typed value provenance, hard effect/permission/compatibility barriers, bounded readable synthesis, verifier and staging semantics, and exact finite-sample admission. Each element has predecessors; their composition defines the new safety argument. The statistical component remains the weakest empirically because the observed gate is all-or-none rather than a demonstrated risk-coverage discriminator (section 8).

GCS adds an interface-level specialization over this guarded program: a bounded projection of verified live-outs, a signed task-semantic argument contract, and an exact continuation pin allow the region to execute before the first provider request. The individual ideas are not new—manual macros, projection pushdown, and partial evaluation are standard—but their composition preserves the compiler’s internal effect checks, provenance, staging, and fallback rather than treating a generated meta-tool as an opaque trusted shortcut. The 12-pair result supports that mechanism on one family. The later six-pair comparison finds structural parity with an independently authored pre-model program, so the scientific claim is automatic guarded specialization, not superior runtime efficiency once a human has already placed the same program correctly.

The portfolio adds a second, narrower composition: selection across transformation *classes* rather than configurations within one declared graph, with separate exact bounds on task failure and non-positive paired utility, compatibility invalidation, and a human-review deployment mode. FlowCompile already constructs reusable accuracy-latency configuration sets and downstream routing; GEPA evolves residual prompts; AWO and Agent JIT generate reusable or task-specific execution structures. We therefore do not claim that portfolio search itself is new. The distinct object here is an evidence-gated choice among baseline, compiler, and reviewed application-interface redesign. Its current evidence is only one family and does not demonstrate that this choice rule beats a fixed macro policy.

The NESTFUL result is as important as the live saving. A system that emitted all 32 recurrent families would look more productive, yet none has the 92 zero-violation

group records required by the configured calculation. Treating those records as independent is a load-bearing sampling assumption, not a property established by the benchmark. Refusal exposes the data requirement rather than moving it into an undocumented heuristic.

7.2 Engineering implications

First, compile-time cost must be amortized. A stable, high-volume prefix can repay 116 discovery traces; a low-volume workflow may never do so. Second, compaction and prompt caching interact: deleting turns changes the number and shape of cache reads/writes, so token reduction need not equal cost reduction. Third, effect declarations are part of the trusted computing base. A misdeclared external write cannot be repaired by statistics. Fourth, the runtime-position key must become explicit before suffix regions are supported. The pilot demonstrates that a compiler and runtime must share the same control-point semantics. Fifth, a task-specific interface and its execution position are separate optimizations: merely exposing a macro still leaves a model turn to select it, whereas a continuation-pinned pre-model composite can remove that turn. Sixth, learned-optimizer overhead needs its own ledger. In the bounded GEPA study the optimization spends 59 provider requests and selects no change, so there is no deployment benefit against which that cost can be amortized. Seventh, benchmark breadth cannot be summarized by a leaderboard average. The ten-source audit finds gold plans, partial fixtures, encrypted questions, real issues, simulated state, and complete recorded traces; a compiler must preserve those substrate distinctions or it will confuse missing evidence with failure.

7.3 Extension path

Two of these follow directly from section 5.4. First, the selective objective should price *cache-prefix fragmentation*: a route whose traffic share is too small to amortize its own cache write is a net loss even when its prompt is strictly shorter, and eq. (5) currently counts only tokens and calls. Second, the break-even in section 5.3 should be computed against the *cached* baseline price rather than the list input price, so that a workload whose baseline is already cache-dominated is reported as having little dollar headroom before a compiler is built for it.

Portfolio optimization beyond the pilot. The implemented portfolio layer selects among measured actions and can recommend a macro for review; it does not synthesize arbitrary macro code, execute cache-only or model-routing candidates, estimate developer effort, or learn a policy across workflow families. The next experiment must include families deliberately spanning stable bundles, branching prefixes, cache-dominated routes, unsafe effects, and low-support abstention. It should measure construction, review, maintenance, invalidation, and drift costs and compare against always-macro, always-compile, cache-only, FlowCompile-style configuration selection, and a learned contextual policy. Only then can the central selection claim be stronger than “the pilot correctly chose the already stronger macro.”

The reusable library boundary is the typed Episode IR plus an optimization-pass API. GCS now covers the narrow read-only interface case described in section 3.3. Remaining near-term extensions include: (1) e-graph or counterexample-guided synthesis within the same closed semantic library; (2) global familywise risk allocation across multiple artifacts; (3) transactional compilation of idempotent writes with approval bar-

riers; (4) explicit execution-plan scheduling to combine compaction with LLMCompiler-style parallelism; (5) cost-aware Pareto artifact selection inspired by FlowCompile; (6) trace-driven prompt/tool-surface and memory compaction evaluated behind the same gates; and (7) adapters for other orchestration frameworks. None is claimed as implemented.

8 Limitations and Threats to Validity

The natural task still constrains the evidence shape. The original conformance prompt names three tools in an exact order, so its recurrence is constructed. The natural follow-ups remove tool names and order, and compiler eligibility no longer requires an exact trace, but the requested title, state, labels, and evidence excerpt still make the three sources useful. All 132 expanded discovery agents (and all 80 in the earlier study) converge on one order. This is stronger evidence of naturally selected recurrence than the original study, but it does not establish discovery in open-ended plans where tools may be skipped, substituted, branched, or revisited for reasons not fixed by an output schema.

Factual quality is narrow and depth-sensitive. The original five-boolean oracle does not check summary factuality and accepts fluent fabrication. The natural studies replace it with exact snapshot fields and source-supported excerpts scored independently of tool order, but this remains an automatic extractive contract rather than human semantic adjudication. The expanded two-read artifact passes 30/30 while the earlier three-read artifact catches one compiled-only Markdown-link alteration. Thus preservation is observed at one depth and explicitly unsupported at the other. Both audited oracle corrections show that even strict-looking metrics can drift; raw online labels and correction records are retained. In the earlier run the corrected eligibility rule would swap one discovery trace, but the evaluated artifact remains the one actually executed. In the expanded run the semantic regrade changes only literal tool-contract labels, not compiler inputs, provider outputs, or measurements. The post-hoc continuation replay repairs the retained exact-source miss with a task-specific deterministic renderer, but it is not a live evaluation and does not replace human semantic adjudication or a separately calibrated continuation-risk frontier.

The selective gate did not discriminate in this run. On the sealed artifact, every grid threshold below 0.14 admits zero calibration groups and every threshold at or above 0.14 admits all 92, with zero violations throughout. The score therefore behaved as an all-or-none support test rather than as a ranking of safe against unsafe inputs, and six of its seven feature weights are exactly zero (only `hull_margin` is non-zero).

The cause is diagnosable rather than mysterious, and it is a property of the data, not of the estimator. The score is fitted on development-group *unproductive* outcomes, and the artifact passed all eight development replays — so the logistic model was fitted with *zero* positive examples on eight observations over seven features. A single-class fit yields a near-constant score, and a near-constant score is exactly what produces the observed step at $\eta = 0.14$: all 92 calibration entries land in one interval. Reporting the positive count is therefore mandatory for any such gate, and we report it here as zero. Two honest routes exist: build a development set containing genuine unproductive outcomes and publish a risk-coverage curve at several α , or replace the learned score with an explicitly one-class construction (distance-to-hull, or a conformal nonconformity measure

over entry features) and describe it as such. Until one of those is done, contribution (v) is demonstrated only as a sample-size counter: on NESTFUL the gate refuses everything because $26 < 92$, and here it admits everything because the score is constant. We cannot claim a demonstrated risk-coverage frontier from this evidence. Establishing one needs calibration and test sets containing natural error: hard-but-supported inputs, out-of-domain entries, source and schema drift, stale versions, ambiguous provenance, and partial tool failures.

Both natural-order artifacts repeat the same pattern. In the earlier run, thresholds below 0.14 admit no calibration records and those at or above 0.14 admit all 45, with zero registered violations and upper bound 0.0992. In the expanded replication the step moves to 0.11, below which none of 92 records are admitted and at or above which all are, with zero registered violations and upper bound 0.0498. Two fitted weights are non-zero, but the observed admission decision is still all-or-none. Natural planning and more calibration data therefore do not repair the missing risk-coverage discrimination.

The original latency result is confounded by condition order. The fixed-prefix harness runs the entire baseline batch and then the entire compiled batch, without randomizing or counterbalancing order within pairs. Provider load, connection reuse, and prompt-cache warmth therefore differ systematically between conditions. The request-count reduction is structural and unaffected, but the -85.0% wall-latency figure should be read as an observation under this ordering rather than a clean causal estimate; the same confound is the most likely explanation for the refusing conditions of section 5.4 billing more than their baselines on identical token counts. Both natural-order studies counterbalance all six three-condition permutations; the expanded run assigns each exactly five primary records. This removes ordinal imbalance but cannot eliminate provider noise or cache interference.

Removing model turns can remove guardrail evaluations. Moderation, policy, and guardrail checks in agent frameworks commonly run *at* model boundaries. A region that deletes three of four boundaries therefore risks deleting three of four guardrail evaluations, and neither our effect catalog nor the read-only policy covers that: a guardrail is not a tool call, so it is invisible to both. Our demonstrations expose no guardrails, so the gap is untested rather than benign. A deployment must either re-run guardrails inside the permission facade for every elided boundary, or declare guardrail presence an explicit barrier condition alongside handoffs and approvals. We recommend the latter until the former is implemented and tested, and note that a misdeclared external write remains undetectable by statistics — detecting it needs differential execution in a sandbox or declaration-versus-observation diffing, neither of which we implement.

Registry integrity is configured off in the reported run. Artifact records are mutable Python objects, the reported experiment sets lifecycle and approval fields directly, and the archived artifact carries an empty signature because signing was not enabled. The immutability and signature-verification semantics described in section 3 are therefore capabilities of the registry, not properties this experiment demonstrates.

Empirical scope. The confirmatory compiler study contains 30 primary issues, the prospective portfolio pilot contains 12, the exploratory GCS comparison contains 12

further pairs, and the learned/manual comparator contains six five-condition blocks, all from one repository/domain and one model configuration. The GCS cohort contains only bug and other-labelled issues; the comparator contains bug, enhancement, and other but no question-labelled issue. It uses a frozen public snapshot, not the live GitHub API, so network/service behavior, concurrent mutation, rate limits, and authentication are absent. The two exact gates have 92 and 45 calibration records, while the portfolio has only 30 selection groups and uses a looser 15% risk limit. The fresh portfolio test contains no question-labelled issue after strict prior-record exclusion. There is no human study of productivity or user experience and no multi-domain production canary.

The artifact includes a prospective three-domain extension harness for vulnerability evidence, SEC filing facts, and privacy-modified public HMDA records. Provider-free preflight retains 420 real vulnerability groups and 420 real HMDA groups, each passing independently implemented exact-gold reconstruction. The SEC pool is absent because the required compliant source contact is not configured; consequently the protocol is not frozen, human macro approvals are absent, and no provider execution has run. These artifacts demonstrate data-pipeline and control-plane feasibility only and are excluded from every effectiveness result in this paper.

Baseline scope. The natural study runs both the unchanged SDK loop and a live hand-written composite tool. In the expanded partial-compilation study, the macro and compiler each use two provider requests and pass 30/30, but the macro uses one local tool call rather than three and is better on tokens and estimated cost. In the earlier aggressive study, GAC saves one more provider request and more tokens but records a factual miss. The offline suite independently gives the macro the lower request ratio on three of five workloads. The compiler itself remains compile-or-retain; the new portfolio layer selects the measured macro and emits a review-required recommendation. Separately, GCS packages an admitted read program behind a synthesized projection and beats the measured provider-visible macro on 12 exploratory pairs; it does not synthesize arbitrary application logic. The six-pair follow-up gives an independently authored program the same pre-model position and finds equal requests, interfaces, input tokens, and exact quality. Official GEPA retains its seed under a 14-task-evaluation, three-reflection budget. The portfolio does not generate the macro, and on a single family its decision remains indistinguishable from an always-macro rule. AWO, AWM, Agent JIT, EvoC2F, LLMCompiler, FlowCompile, and plan caching remain unexecuted. The evidence establishes an intervention and engineering trade-off, not state-of-the-art superiority.

Portfolio scope. The weighted utility is a declared operator preference, not a universal scientific metric. Its current dimensions omit construction time, review effort, maintenance, drift, cache fragmentation, and authorization changes. Both measured actions pass both binary gates, so exact risk controls admission but does not explain the choice; the utility weights do. The prospective result validates the chosen arm against baseline, not the selector against alternative selection algorithms. A claim that the portfolio advances the state of the art requires multi-family decisions, closer learned baselines, sensitivity to the weights, and time-forward evaluation under drift.

Statistical scope. The pilot and final cohort are separated, but the study was not externally preregistered. Secondary tests are unadjusted and provider latency is noisy.

The optimizer comparison contains only six pairs; its exact signed-rank minimum is therefore .03125, and one provider-span record fails the host-wall consistency check. Clopper–Pearson is exact but conservative. The gate’s guarantee applies to the recorded group-level violation definition only when admitted group indicators are i.i.d. or conditionally i.i.d.; drift, clustered tenants, adaptive artifact selection, or incomplete outcomes invalidate a naive interpretation. Each artifact is calibrated individually; the implementation lacks a global guarantee when many artifacts are selected.

Semantic scope. Recorded output equality and empirical contracts do not prove full semantic equivalence. The closed DSL intentionally misses legitimate transformations and loops. Tool-effect truth, permissions, freshness, and isolation are application-supplied. The current version is read-only, does not parallelize synthesized calls, and task-semantic argument contracts are application-supplied trusted declarations. The narrow SDK adapter bypasses streaming, hosted tools, MCP, handoffs, and loops.

Reproducibility scope. The original experimental workspace did not retain prior .git history. The public artifact begins from a later initial snapshot, so pre-snapshot commit ancestry and CI state cannot be reconstructed. Dataset revisions and file hashes are pinned, raw results are retained, and the full local suite passes on Python 3.14.4; nevertheless, provider responses and latency will not be byte-identical on rerun. API access and the named model are required to repeat the live study.

9 Conclusion

Guarded agentic compaction treats repeated traces as candidate evidence, not permission to rewrite an agent. It compiles only value-grounded, read-only prefixes that satisfy explicit compatibility, verification, and finite-sample admission requirements; otherwise it keeps the original agent.

The experiments show why each boundary matters. On real public GitHub records, a conservative two-read artifact passes 30/30 exact contracts and halves provider requests, while an aggressive three-read artifact records one factual miss. The handwritten macro also passes 30/30 and is cheaper, so automatic discovery and guarded lifecycle—not universal runtime dominance—are the defensible advantages. GCS removes a provider-visible selection turn on 12 exploratory pairs, but a fairly placed manual program ties it on six later cases. NESTFUL and API-Bank add a different result: despite recurrent, executable traces, every family retires because the configured evidence requirement is unmet. The ten-benchmark audit shows that most popular agent benchmarks do not retain the observed state and results needed to test post-trace compilation at all.

The resulting design principle is simple: evaluate workflow optimizers by the evidence they require, the effects and positions they refuse, and the state to which they fall back—not only by the turns they remove. Stronger claims now require a time-forward, multi-family comparison with natural failures, a non-degenerate risk–coverage frontier, manual construction and maintenance cost, cache effects, drift, and closer executable workflow-learning baselines.

Artifact Availability and Ethical Considerations

The artifact package includes scripts, pinned-source manifests, raw JSON/CSV results, generated figures/tables, a reproduction guide, and a critical self-review. The GitHub

records are already public and distributed through an Apache-2.0 dataset snapshot; the paper stores issue identifiers and derived task outputs but no API secrets. The approach should not be used to bypass approvals or automate regulated decisions. Its current read-only effect policy and fallback semantics are deliberate safeguards, not guarantees against a dishonest effect catalog.

References

- [1] S. Abuzakuk, A.-M. Kermarrec, R. Sharma, R. M. Veski, and M. de Vos. Optimizing agentic workflows using meta-tools. *arXiv preprint arXiv:2601.22037*, 2026. URL <https://arxiv.org/abs/2601.22037>.
- [2] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2507.19457>. Oral presentation.
- [3] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei. Learn then test: Calibrating predictive algorithms to achieve risk control. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=TNc0rox3tQ>.
- [4] V. Bala, E. Duesterwald, and S. Banerjia. Dynamo: A transparent dynamic optimization system. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 1–12, 2000. doi: 10.1145/349299.349303.
- [5] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025. URL <https://arxiv.org/abs/2506.07982>.
- [6] K. Basu, I. Abdelaziz, K. Kate, M. Agarwal, M. Crouse, Y. Rizk, K. Bradford, A. Munawar, S. Kumaravel, S. Goyal, X. Wang, L. A. Lastras, and P. Kapanipathi. NESTFUL: A benchmark for evaluating LLMs on nested sequences of API calls. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33538–33547, 2025. doi: 10.18653/v1/2025.emnlp-main.1702. URL <https://aclanthology.org/2025.emnlp-main.1702/>.
- [7] S. Bates, A. Angelopoulos, L. Lei, J. Malik, and M. I. Jordan. Distribution-free, risk-controlling prediction sets. *Journal of the ACM*, 68(6), 2021. doi: 10.1145/3478535. URL <https://arxiv.org/abs/2101.02703>.
- [8] C. F. Bolz, A. Cuni, M. Fijałkowski, and A. Rigo. Tracing the meta-level: PyPy’s tracing JIT compiler. In *Proceedings of the 4th Workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems (ICOOOLPS)*, pages 18–25, 2009. doi: 10.1145/1565824.1565827.
- [9] Y. Chen, H. Peng, J. Liu, Z. Wen, D. Liu, and L. Zhang. AgentSlimming: Towards efficient and cost-aware multi-agent systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 30064–30086, 2026. doi: 10.18653/v1/2026.acl-long.1387. URL <https://aclanthology.org/2026.acl-long.1387/>.

-
- [10] J. Cheney, L. Chiticariu, and W.-C. Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009. doi: 10.1561/19000000006.
- [11] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970. doi: 10.1109/TIT.1970.1054406.
- [12] J. Chundru. Agentic compilation: Mitigating the LLM rerun crisis for minimized-inference-cost web automation, 2026. URL <https://arxiv.org/abs/2604.09718>.
- [13] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404.
- [14] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993. doi: 10.1201/9780429246593.
- [15] M. D. Ernst, J. Cockrell, W. G. Griswold, and D. Notkin. Dynamically discovering likely program invariants to support program evolution. *IEEE Transactions on Software Engineering*, 27(2):99–123, 2001. doi: 10.1109/32.908957.
- [16] A. Gal, B. Eich, M. Shaver, D. Anderson, D. Mandelin, M. R. Haghighat, B. Kaplan, G. Hoare, B. Zbarsky, J. Orendorff, J. Ruderman, E. W. Smith, R. Reitmaier, M. Bebenita, M. Chang, and M. Franz. Trace-based just-in-time type specialization for dynamic languages. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 465–478, 2009. doi: 10.1145/1542476.1542528.
- [17] S. Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 317–330, 2011. doi: 10.1145/1926385.1926423.
- [18] U. Hölzle, C. Chambers, and D. Ungar. Debugging optimized code with dynamic deoptimization. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 32–43, 1992. doi: 10.1145/143095.143114.
- [19] Hugging Face. Github issues dataset card. Hugging Face Datasets, 2025. URL <https://huggingface.co/datasets/helmo/github-issues>. Revision e344be7b84d199661a9956036991e1fc25715a47.
- [20] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu. LLMlingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://aclanthology.org/2023.emnlp-main.825/>.
- [21] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.

-
- [22] N. D. Jones, C. K. Gomard, and P. Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993. URL <https://www.itu.dk/people/sestoft/pebook/>.
- [23] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=sY5N0zY50d>.
- [24] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami. An LLM compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 24370–24391, 2024. URL <https://proceedings.mlr.press/v235/kim24y.html>.
- [25] J. Li, Z.-W. Hong, M. Shen, Y. Zhang, and C. Gan. FlowCompile: An optimizing compiler for structured LLM workflows. *arXiv preprint arXiv:2605.13647*, 2026. URL <https://arxiv.org/abs/2605.13647>.
- [26] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li. API-bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187. URL <https://aclanthology.org/2023.emnlp-main.187/>.
- [27] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023. URL <https://arxiv.org/abs/2308.03688>.
- [28] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, F. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, and R. Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025. doi: 10.18653/v1/2025.findings-naacl.65. URL <https://aclanthology.org/2025.findings-naacl.65/>.
- [29] J. M. Lucassen and D. K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 47–57, 1988. doi: 10.1145/73560.73564.
- [30] E. Lumer, F. Nizar, A. Jangiti, K. Frank, A. Gulati, M. Phadate, and V. K. Subbiah. Don’t break the cache: An evaluation of prompt caching for long-horizon agentic tasks, 2026. URL <https://arxiv.org/abs/2601.06007>.
- [31] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12:153–157, 1947. doi: 10.1007/BF02295996.
- [32] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom. GAIA: A benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2023. URL <https://arxiv.org/abs/2311.12983>.

-
- [33] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html.
- [34] OpenAI. Agents SDK guide. OpenAI Developer Documentation, 2026. URL <https://developers.openai.com/api/docs/guides/agents>. Accessed 2026-08-03.
- [35] OpenAI. API pricing. OpenAI Developer Documentation, 2026. URL <https://developers.openai.com/api/docs/pricing>. Accessed 2026-08-03.
- [36] K. Opsahl-Ong, M. J. Ryan, J. Purtell, M. Zaharia, and O. Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. *arXiv preprint arXiv:2406.11695*, 2024. URL <https://arxiv.org/abs/2406.11695>.
- [37] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392, 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- [38] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023. URL <https://arxiv.org/abs/2307.16789>.
- [39] A. Solar-Lezama, L. Tancau, R. Bodik, S. Seshia, and V. Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 404–415, 2006. doi: 10.1145/1168857.1168907.
- [40] Y. Song. Cache-aware prompt compression: A two-tier cost model for LLM API caching, 2026. URL <https://arxiv.org/abs/2607.15516>.
- [41] W. M. P. van der Aalst. *Process Mining: Data Science in Action*. Springer, 2 edition, 2016. doi: 10.1007/978-3-662-49851-4.
- [42] J. Wang, M. Zheng, and T. Wei. COVENANT: Natural-language workflow compilation for aligned agent execution. *arXiv preprint arXiv:2607.25400*, 2026. URL <https://arxiv.org/abs/2607.25400>.
- [43] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911, 2025. URL <https://proceedings.mlr.press/v267/wang25bx.html>.
- [44] J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025. URL <https://arxiv.org/abs/2504.12516>.

- [45] L. Wei, Q. Liu, R. Huang, X. Peng, S. Xie, L. Lin, C. Jiang, Y. Xu, T. Yang, J. Liu, L. Cai, Z. Kang, and B. Wang. EvoC2F: Compiling tool orchestration for efficient and evolvable LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL <https://openreview.net/forum?id=ZSGB91kM0G>.
- [46] F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6): 80–83, 1945. doi: 10.2307/3001968.
- [47] C. Winston, R. Y. Wang, A. Mirhoseini, and C. Kozyrakis. Agent JIT compilation for latency-optimizing web agent planning and scheduling. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL <https://arxiv.org/abs/2605.21470>.
- [48] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- [49] L. Yue, K. R. Bhandari, C.-Y. Ko, D. Patel, S. Lin, N. Zhou, J. Gao, P.-Y. Chen, and S. Pan. From static templates to dynamic runtime graphs: A survey of workflow optimization for LLM agents, 2026. URL <https://arxiv.org/abs/2603.22386>.
- [50] Q. Zhang, M. Wornow, and K. Olukotun. Cost-efficient serving of LLM agents via test-time plan caching. *arXiv preprint arXiv:2506.14852*, 2025. URL <https://arxiv.org/abs/2506.14852>.
- [51] X. Zhou, J. Liu, L. Li, R. A. Rossi, and X. Hu. Counterfactual trace auditing of LLM agent skills, 2026. URL <https://arxiv.org/abs/2605.11946>.

A Reproducibility Details

A.1 Environment and sealed evidence

The live run manifest records macOS 26.5.2 arm64, Python 3.14.4, OpenAI Agents SDK 0.19.2, OpenAI Python 2.52.0, gpt-5.6-luna, low reasoning effort, and provider default temperature because the selected model rejected an explicit temperature parameter. The result explicitly records that the OpenAI key and Hugging Face token were used and that no secret was serialized. The experimental workspace did not retain prior Git history; the public repository starts from a later initial snapshot.

The GitHub and NESTFUL revisions begin e344be7b84d1 and fc2c4123e735, respectively. Their complete 40-character identifiers, exact file digests, and byte counts are in the two `source_manifest.json` files. Generated artifact digests are in the artifact manifest under `paper/results`.

A.2 Commands

The provider-free reproduction is:

```
.venv/bin/python paper/scripts/nestful_benchmark.py
R=paper/results/github_natural_replication
.venv/bin/python paper/scripts/github_live_study.py \
    --task-design natural-extractive-v2 \
```

```

--regrade-results \
--results-path "$R/results.json"
.venv/bin/python paper/scripts/build_artifacts.py
O=paper/results/optimizer_head_to_head
H=paper/scripts/github_optimizer_head_to_head.py
.venv/bin/python "$H" \
--regrade-existing "$O/results.json"
M=paper/results/multidomain/preflight
V=paper/scripts/validate_multidomain.py
.venv/bin/python "$V" \
--pool vulnerability="$M/vulnerability" \
--pool hmda="$M/hmda" --out "$M/validation.json"
.venv/bin/coverage run -m pytest -q
.venv/bin/python scripts/verify_release.py
TECTONIC --outdir ../build main.tex

```

The live studies are intentionally separate because they incur cost. The expanded natural-order replication, which supplies the paper's 30-pair confirmatory result, used:

```

.venv/bin/python paper/scripts/github_live_study.py \
--task-design natural-extractive-v2 \
--evaluation-order counterbalanced \
--include-macro --discovery-cases 132 \
--train-cases 16 --dev-cases 8 \
--calibration-cases 92 --test-per-class 10 \
--repeat-cases 10 --concurrency 8 \
--seed 20260802

```

The prospective portfolio pilot first seals a decision from that replication and then evaluates the selected, reviewed action on previously unseen public GitHub issues:

```

P=paper/scripts/portfolio_live_study.py
.venv/bin/python "$P" \
--preflight --cases-per-class 4
.venv/bin/python "$P" \
--cases-per-class 4 --approve-reviewed-macro

```

The approval flag records an explicit experimental decision; it does not enable autonomous macro deployment. The script rejects overlap with every earlier study issue and writes native SDK traces plus the exact frozen decision to `results/portfolio_live`. The bounded learned/manual comparator used real public records and real provider calls:

```

H=paper/scripts/github_optimizer_head_to_head.py
.venv/bin/python "$H" \
--train-cases 4 --val-cases 2 --test-cases 6 \
--max-metric-calls 16 --max-proposals 3 \
--reflection-minibatch 2

```

It executes official GEPA 0.1.4. The provider-free `--regrade-existing` command above recomputes grading, comparisons, and measurement validation without changing any

retained answer, trace, token, latency, or cost observation. The earlier aggressive natural-order run used:

```
cd paper/scripts
../../.venv/bin/python \
  github_natural_workflow_study.py \
  --discovery-cases 80 \
  --train-cases 20 --dev-cases 10 \
  --calibration-cases 45 \
  --test-cases 18 --concurrency 6
```

The explicit flags below reproduce the older archived 18-pair, six-repeat prescribed-prefix design; that script's defaults select a larger run:

```
../../.venv/bin/python github_live_study.py \
  --test-per-class 6 --repeat-cases 6
```

It reads OPENAI_API_KEY and optionally HF_TOKEN from .env. The script never prints either value. Use --smoke before a paid full run.

B Implementation Audit

The package contains 19,264 measured Python statements across source, tests, demos, benchmarking, and experiment drivers; the post-fix full suite collects and passes 350 tests at 74.58% statement coverage. pip check and the repository release verifier pass. Core provenance, windows, dispatcher, facade, schemas, registry, and statistics modules have strong focused coverage. Weak areas include external adapter acquisition paths (18.5%), TGWS packaging (27.7%), the outer runner (67.3%), replay (65.5%), and the older live experiment drivers (30–31%). The new prospective portfolio driver is at 54.1%. Coverage is evidence of exercised lines, not correctness.

The review found and corrected a high-severity compiler-runtime mismatch: the miner could emit suffix-only regions while the shipped runtimes resolve at the initial boundary. GrcConfig.prefix_only now defaults to true; the miner records non_prefix_runtime; and a golden-trace regression test verifies that an otherwise eligible suffix is rejected. The archived pilot is retained as direct evidence of the impact.

Residual engineering debt includes per-artifact rather than global risk control, mutable in-memory artifact objects, shared-secret rather than public-key registry signing, non-atomic directory-level registry save, no runtime concurrency scheduler, no supported multi-framework runtime adapters beyond a generic wrapper, and incomplete CI/provenance metadata in this checkout.

C Complete Claims Register

D Additional Numerical Results

The final live discovery run completes 132/132 requested episodes. Tool-contract eligibility is 132/132, task success is 129/132 (97.7%), and category accuracy is 130/132 (98.5%). Discovery consumes 517,185 input and 16,108 output tokens across 528 provider requests. All final paired-test executions complete without recorded runtime failure.

The expanded natural-order replication retains 132 discovery and 120 evaluation outputs with no infrastructure failure. Discovery produces 130/132 exact-source

passes; all 132 traces use the same three-read order, while 130 enter the compiler-eligibility pool and 116 enter the 16/8/92 split. The full three-read candidate is rejected because the comment limit is ungroundable; the selected two-read program removes two provider requests. Across 30 primary pairs, its mean compiled-minus-baseline differences are: input tokens -1626.8 $[-1716.2, -1541.5]$, output tokens -56.1 $[-65.2, -47.8]$, total tokens -1682.9 $[-1772.5, -1596.7]$, wall latency -3.187 s $[-3.758, -2.660]$, and estimated cost $-\$0.000279$ $[-\$0.000313, -\$0.000241]$. Requests change from four to two and tool calls remain three. The macro also uses two requests but one tool call, 1,780.8 tokens, 3.25 seconds, and $\$0.000545$ per issue, versus 2,576.5 tokens, 2.97 seconds, and $\$0.000593$ for the partial compiler. All three primary arms pass 30/30 exact-source and full task contracts. These are 10,000-sample paired bootstrap intervals; secondary tests are unadjusted.

For the 18 primary pairs, the mean compiled-minus-baseline differences and 95% paired bootstrap intervals are: input tokens -2503.8 $[-2749.4, -2271.4]$, output tokens -68.8 $[-72.1, -66.0]$, total tokens -2572.6 $[-2816.6, -2340.2]$, provider latency -8.743 s $[-11.741, -6.139]$, wall latency -8.744 s $[-11.742, -6.140]$, and estimated cost $-\$0.000412$ $[-\$0.000473, -\$0.000348]$. Tool-call difference is exactly zero.

NESTFUL blocked-window counters are not mutually exclusive episode counts because the miner enumerates multiple candidate spans: 10,172 ambiguous-slot spans, 5,341 spans with live-ins outside the entry schema, 4,853 partial runs, and one ungrounded-slot span. The 12 synthesized families contain programs of two to five steps. The 20 failed family attempts split evenly between unsupported loop-predicate synthesis and grouped-refit grounding failure.

The portfolio selector uses the 30 independent replication issues, treating repeated conditions within an issue as one group. At 95% simultaneous confidence with separate 15% quality-risk and regret-risk limits, both the compiler and macro have zero observed violations and exact upper bounds of 0.136; their mean utilities are 0.327 and 0.489, respectively. The macro is therefore recommended for human review. On 12 fresh issues, baseline and macro both pass 12/12 exact-source, factual, and tool-contract checks. The macro reduces provider requests by 50.0%, tool calls by 66.7%, total tokens by 59.2%, wall latency by 71.6%, and estimated cost by 40.6%. Paired two-sided Wilcoxon tests give $p = 0.000488$ for tokens, latency, and cost. This is a prospective within-family result, not evidence that selection outperforms an always-macro policy across heterogeneous workflows.

The bounded optimizer comparison uses four optimization-training, two validation, and six deployment issues, all disjoint from each other and every earlier cohort. Every deployment condition passes 6/6 exact factual and tool contracts. Unchanged, GEPA, GCS, GCS plus the retained GEPA seed, and manual pre-model execution use respectively 4, 4, 1, 1, and 1 mean provider requests. Mean input tokens are 3,542.0, 3,530.0, 770.8, 770.8, and 770.8. GCS and manual execution therefore tie structurally; their paired latency and estimated cost differences are not significant. GEPA retains its seed after 14 task evaluations and three reflections. Its separate optimization ledger records 59 provider requests, 63,954 tokens, and $\$0.01162959$ estimated cost. One provider-span latency exceeds the surrounding host wall time and is retained but excluded from provider-span comparisons.

E Benchmark Selection Rationale

The extension now gives all ten named benchmark families an explicit path. NESTFUL and API-Bank execute the compiler because they preserve complete values. BFCL executes 200 gold plans through its official checker. ToolSandbox, the maintained τ^2/τ^3 implementation, and BrowseComp execute bounded official paths with real provider calls. ToolBench and AgentBench execute source adapters but remain full-evaluation gated by unversioned external data, live backends, and services. SWE-bench Verified's 500 real issues are normalized, while its container execution is host-gated. GAIA remains authorization gated. No missing path receives an imputed metric.

These integrations do not make the benchmarks interchangeable. A gold plan lacks observed tool results; an encrypted web question lacks a replay-safe local search trace; a stateful simulator can measure the agent but is not a real-world deployment; a real issue without an agent trajectory tests generation but not post-trace compilation. A future compiler benchmark should record entry snapshots, full argument/result payloads, effect and permission labels, independent grouping keys, commit boundaries, task outcomes, and a frozen paired protocol. Without these fields, a benchmark can measure tool use but cannot substantiate safe trace-derived replacement.

E.1 Reproduction commands for the all-source audit

Source acquisition writes into a caller-provided disposable directory and serializes only revisions, hashes, and gate reasons. After acquisition, the provider-free matrix and the API-Bank/BFCL paths are rebuilt with:

```
SCRIPTS=paper/scripts
OUT=paper/results/external_benchmarks
python "$SCRIPTS/external_benchmark_sources.py" \
  --root "$SOURCE_ROOT" \
  --output "$OUT/source_preflight.json"
python "$SCRIPTS/external_benchmark_matrix.py" \
  --source-root "$SOURCE_ROOT"
python "$SCRIPTS/api_bank_benchmark.py" \
  --source-root "$SOURCE_ROOT"
python "$SCRIPTS/bfcl_structural_benchmark.py" \
  --source-root "$SOURCE_ROOT"
```

ToolSandbox is isolated by the pinned Dockerfile. The τ and BrowseComp live runs require OPENAI_API_KEY; GAIA acquisition additionally checks HF_TOKEN. The sanitizers retain hashes, counts, scores, timing, and usage only. Raw ToolSandbox and τ trajectories remain in disposable upstream run directories, and BrowseComp questions and answers are decrypted only in process memory.

The repository now contains the provider-free portion of such a prospective extension. It checksum-binds 420 real vulnerability groups and 420 privacy-modified public HMDA groups, independently reconstructs exact gold, freezes group/lineage-aware study roles, and supplies hash-chained execution and sealed-analysis controls. SEC acquisition remains gated on a compliant source contact, so the three-domain protocol cannot yet freeze and no provider, quality, latency, cost, or determinism result exists. The retained preflight is feasibility evidence, not a third empirical tier in this manuscript.

F Artifact Lifecycle Boundary

The study artifact passed replay and calibration, then was activated only under paper-protocol-lab-only to measure its runtime path. This does not demonstrate operator approval, shadow/canary health, rollback, or production monitoring. Deployment must preserve shadow → canary → live and attach privacy, tenant-isolation, incident-response, and drift-monitoring evidence.

Table 12. Claim-to-evidence mapping. Unsupported claims are included to prevent later narrative drift.

ID	Claim	Direct evidence	Verdict
C1	The expected producer appears in the provenance candidate set	Pinned NESTFUL: 5,531/5,746 dependency slots	96.3% candidate recall; 80.7% unique resolution
C2	A naturally recurring prefix can remove model decisions	132 live discovery records; 30 counterbalanced real-record tests	Two-read prefix cuts requests 50%; one frozen-snapshot domain
C3	Source-grounded task quality is preserved	Expanded partial compiler: all three arms 30/30; earlier aggressive compiler: 17/18	Supported on the expanded sample, not across domains or compilation depths
C4	Configured selective admission is data hungry	NESTFUL max support 26 vs. required 92	Supported for this grid/risk/confidence setting
C5	Compaction improves text determinism	Exact-answer agreement 1/6 vs. 0/6 ($n = 6$)	Not supported
C6	The artifact is production safe	No live GitHub service, canary, or multi-domain test	Not supported
C7	The learned gate discriminates risky from safe inputs	0 positive dev examples; gate admits none or all	Not supported
C8	Factual summary quality is preserved	Oracle accepts fluent fabrication	Not evaluated
C9	The learned compiler generally dominates a hand-written macro	Macro beats partial GRC on 30 pairs; GCS beats provider-visible macro on 12 later pairs	Not supported across interfaces or workflow families
C10	A separate continuation contract can recover the retained exact-source miss	Provider-free replay detects issue 6602 and checked-renders 1/18 cases	Verified counterfactual; no live latency, cost, or cross-domain claim
C11	The portfolio recommends a measured macro	30 frozen independent groups; 12 fresh paired issues	Verified on one workflow family; recommendation requires human review
C12	The portfolio synthesizes a macro or evaluates a cache action	Public portfolio API accepts externally supplied measurements only	Not implemented
C13	Portfolio selection beats an always-macro policy across workflows	One family in which the measured macro is selected	Not supported
C14	Guarded composite synthesis beats the measured provider-visible macro	12 fresh paired real-record cases; both 12/12 exact	Exploratory: requests -50%, tokens -38.9%, latency -40.0%, cost -32.3%; one workflow family
C15	GCS outperforms an equally pre-executed manual program	6 fresh paired cases; both 6/6 exact with 1 request, 1 interface, and identical input tokens	Not supported; structural parity, with no significant cost or latency difference
C16	Bounded GEPA improves this workflow	Official GEPA 0.1.4; 14 task evaluations and 3 reflection calls	Not supported; seed retained, deployment requests unchanged, optimization overhead reported separately